

TWO'S COMPLEMENT FLOATING POINT ARITHMETIC SYSTEM

Provisional Patent Application

Inventor: Nick Spiker

May 16, 2025

ABSTRACT

The disclosed system presents a fundamental reimagining of floating-point arithmetic by employing two's complement representation thruout the entire calculation pipeline. Unlike IEEE-754 which employs sign-magnitude representation with a separate sign bit, the system uses left-aligned normalized fractions with unbiased exponents to create a continuous numerical framework for both real and complex numbers.

The system maintains mathematical identities by implementing a unified approach to numerical states: normal values use standard exponents with N-1 normalized fractions, while all non-normal states (zero, infinity, escaped values, undefined conditions) share a single reserved exponent value. These ambiguous states are differentiated by distinct bit patterns in the fraction component - with escaped values (exploded or vanished) preserving phase information, and undefined states encoding their specific cause. This architecture preserves critical mathematical relationships that IEEE-754 violates, such as ensuring that $a \times b = 0$ if and only if $a = 0$ or $b = 0$ (multiplicative identity) and that $a - b = 0$ if and only if $a = b$ (subtractive identity), without requiring denormalized numbers.

Implementation analysis reveals significant efficiency gains: common operations require substantially fewer instructions compared to IEEE-754 implementations. The system introduces direct bitwise operations on floating-point values thru exponent alignment, proper shift operations as exponent adjustments, and a unified approach to complex arithmetic using shared exponents. The parameterized design allows independent selection of fraction and exponent sizes to optimize for either precision or range according to application requirements.

TECHNICAL FIELD

The present invention relates to computer arithmetic operations, specifically floating-point arithmetic implemented using two's complement representation thruout the entire calculation pipeline.

The system addresses fundamental challenges in numerical computing by eliminating separate sign bit handling and employing multiple normalization levels encoded in the most significant bits of fraction components. By maintaining two's complement properties thruout operations, the system significantly reduces branching complexity and preserves mathematical identities across the entire numerical spectrum.

The invention provides precise tracking of numerical states beyond representable boundaries thru specific bit patterns, ensuring operations maintain proper arithmetic relationships at singularities like zero and infinity. The approach enables unified handling of both real and complex numbers, direct bitwise manipulations thru fraction alignment, and customizable precision/range trade-offs thru independent fraction and exponent sizing.

BACKGROUND

Floating-point arithmetic serves as a fundamental component of digital computing, providing a means to represent and manipulate real numbers within the constraints of finite hardware. Since 1985, the IEEE-754 standard has defined the predominant specifications for floating-point representation and operations across computing.

The IEEE-754 standard employs a structure where floating-point numbers consist of three distinct components: a sign bit, an exponent field with bias, and a mantissa (significand) field with implied leading bit for normal values. This representation necessitates specialized handling for the sign bit separate from magnitude components, introducing conditional branches for most arithmetic operations.

Several technical characteristics of the IEEE-754 approach present implementation challenges:

The separate sign bit requires distinct processing paths for different sign combinations. For instance, adding values with the same sign employs addition logic followed by potential sign adjustment, while adding values with opposite signs requires subtraction logic, possible zero crossing detection, and conditional sign adjustment. These different paths significantly increase the complexity of hardware implementation and potential for pipeline stalls in modern processors.

IEEE-754 defines several special values including positive and negative zero, positive and negative infinity, quiet NaN, and signaling NaN, each requiring explicit detection and specialized handling. When operations produce NaN results, information about what caused the invalid operation is not preserved.

Certain mathematical properties face compromise in the IEEE-754 framework. For example, the multiplication of non-zero values can produce zero or infinity when underflow or overflow occurs, and values that are mathematically unequal may produce a difference of zero if their difference falls below representable magnitude. The latter historically has been addressed with subnormal (denormalized) numbers, which introduce additional performance and implementation complexities.

Particularly problematic is IEEE-754's violation of fundamental mathematical identities, such as the multiplicative zero property. In standard mathematics, $a \times b = 0$ if and only if at least one of a or b equals zero. However, in IEEE-754, when a value underflows past the smallest representable magnitude, it typically gets flushed to zero, causing non-zero values to produce zero when multiplied. This inconsistency can lead to unexpected algorithm behavior and numerical instabilities. Similarly, non-infinite values that overflow can produce infinity, breaking the identity that adding or multiplying finite values should produce non-infinite results.

The fixed allocation of bits between exponent and mantissa in standard IEEE-754 formats such as binary32 and binary64 lacks adaptability for applications with divergent precision and range requirements.

Alternative approaches to floating-point numbers have emerged to address various aspects of these limitations, but all maintain the separation between sign and magnitude, perpetuating the need for explicit sign testing and divergent execution paths for different sign combinations. None provide a comprehensive approach that unifies sign and magnitude thru two's complement representation thruout the entire calculation pipeline.

SUMMARY OF THE INVENTION

The present invention implements a floating-point arithmetic system that employs two's complement representation thruout the entire calculation pipeline. This approach redefines how numerical values are represented and manipulated in computing systems, addressing several technical challenges present in conventional implementations.

The system represents numbers using two primary components: a normalized two's complement fraction and an unbiased two's complement exponent. Unlike systems that use a separate sign bit with magnitude representation, this approach leverages the inherent properties of two's complement arithmetic to simplify operations while expanding floating-point capabilities.

The system implements the following technical mechanisms:

A normalization scheme with multiple levels encoded in the leading bits of the fraction component. Normal numbers utilize N-1 normalization (patterns 01... or 10... in most significant bits) to retain sign information within the two's complement phase. The N-1 normalization retains one bit of sign information (0 for positive, 1 for negative) followed by an opposite bit, providing easy detection of fraction sign when desired.

A single reserved exponent value (AMBIGUOUS_EXPONENT) that identifies values outside normal representation, including escaped values ("exploded" when exceeding maximum magnitude or "vanished" when below minimum magnitude), Zero, Infinity, and multiple undefined states.

Specific bit patterns within the fraction component that indicate different numerical states. These patterns are organized hierarchically, with Zero, Infinity, normal/exploded, vanished, and different categories of undefined values each having distinct patterns. This approach enables efficient state detection and tracking thruout calculations.

A mechanism for preserving phase/orientation information when values exceed normal representation range. Unlike IEEE-754, which truncates to infinity or zero in overflow/underflow conditions, the system transitions to exploded or vanished states that maintain sign/directional information, allowing continued participation in absolute operations like multiplication and division.

Independent selection of fraction and exponent sizes, enabling applications to configure precision and range characteristics according to specific requirements without requiring separate implementations.

A unified representation for both real (Scalar) and complex (Circle) values, where complex numbers use real and imaginary fractions sharing a common exponent to maintain binary fraction alignment and simplify calculations.

Direct bitwise operations on floating-point values thru exponent alignment before applying bitwise logic, preserving mathematical significance thruout operations. Bit shifts are implemented as checked exponent adjustments rather than direct bit manipulation.

A mechanism that preserves operation cause in undefined states, providing specific bit patterns that indicate which operation led to the undefined result (e.g., division by zero vs. infinity minus one), enabling precise error tracing.

By maintaining two's complement properties thruout the calculation pipeline, the system preserves mathematical identities without requiring special case handling, significantly reducing implementation complexity while improving numerical consistency across the entire representable range.

DEFINITIONS

As used herein, the following terms shall have the following meanings, unless the context clearly indicates otherwise:

[0001] "Two's complement representation" refers to a method of representing positive and negative integers where the most significant bit (leftmost) can be checked to determine the sign (0 for positive, 1 for negative). Negative values are encoded by taking the bit complement (logical NOT) of the absolute value and adding one. In this invention, left normalized two's complement representation is applied to floating-point components thruout the entire calculation pipeline, eliminating the sign dependent handling steps for all basic operations (add, subtract, multiply, divide).

[0002] "Scalar" refers to a data type representing real numbers using two primary components: a left normalized two's complement fraction and an unbiased two's compliment exponent. Unlike traditional floating-point formats with separate sign bit, the Scalar's sign information is naturally encoded within the fraction.

[0003] "Circle" refers to a data type representing complex numbers using three components: a real fraction, an imaginary fraction, and a shared exponent (all in left normalized two's complement representation). The shared exponent ensures that both components maintain relative magnitudes, simplifying complex arithmetic operations.

[0004] "Fraction" refers to the component of a Scalar or Circle that stores the normalized significand in two's complement format. The size of the fraction determines the precision of the representation, and its most significant bits encode state information thru normalization levels. Fraction sizes are denoted in shorthand powers of two for convenience and indicated as F3, F4, F5, F6, or F7, corresponding to 8-bit (2^3), 16-bit (2^4), 32-bit (2^5), 64-bit (2^6), and 128-bit (2^7) integer representations respectively.

[0005] "Exponent" refers to the component of a Scalar or Circle that determines the scale of the value. The size of the exponent determines the representable range. Unlike traditional floating-point formats that use biased exponents, this invention uses unbiased two's complement exponents. Exponent sizes are denoted in powers of two and indicated as E3, E4, E5, E6, or E7, corresponding to 8-bit (2^3), 16-bit (2^4), 32-bit (2^5), 64-bit (2^6), and 128-bit (2^7) integer representations respectively.

[0006] "FE Type alias" refers to a shorthand notation for specifying both fraction and exponent bit sizes. For example, ScalarF5E4 represents a Scalar with 32-bit (2^5) fraction and 16-bit (2^4) exponent, equivalent to Scalar<i32, i16> in the Rust language.

[0007] "Phase" or "Orientation" refers to sign information in Scalars or complex angle in Circles. This information is preserved when values escape normal representation, allowing continued absolute operations with mathematically consistent results.

[0008] "Normalization level" or "N-level" refers to the leading same bit count in the fraction, which is used to encode the state of a value (normal, exploded, vanished, Zero, undefined, etc.). The invention implements multiple normalization levels (N-0, N-1, N-2, N-3, etc.) to represent different numerical states with unique bit patterns.

[0009] "Normal value" refers to a Scalar or Circle with a non-zero known magnitude. The fraction(s) are normalized to level N-1 (with bit pattern 01... or 10... in the most significant positions) and an exponent within representable range (not ambiguous). Normal values can directly participate in all arithmetic operations.

[0010] "Ambiguous value" refers to any value with an exponent equal to `AMBIGUOUS_EXPONENT` that indicates the value may require special handling. This includes zero, infinity, exploded, vanished, and undefined states.

[0011] "AMBIGUOUS_EXPONENT" refers to the specific reserved exponent value (one followed by all zeros, e.g., 10000000...) that indicates an ambiguous state. It serves as a single consistent marker for all states outside normal representation.

"Escaped value" refers to a value that has exceeded normal representation range but maintains its phase/orientation information. Unlike IEEE-754, which truncates to infinity or zero, escaped values preserve directional information even though their magnitude is no longer representable. The system implements two types of escaped values: exploded (exceeding maximum magnitude) and vanished (below minimum magnitude).

[0012] "Exploded" refers to a value that has exceeded the maximum representable magnitude but maintains its phase/orientation information. Exploded values are represented by a fraction with N-1 normalization and an ambiguous exponent, allowing continued absolute operations (approaching but not infinity).

[0013] "Vanished" refers to a value that has become too small for normal representation but maintains its phase/orientation information. Vanished values are represented by a fraction with N-2 normalization and an ambiguous exponent (approaching but not zero).

[0014] "Transfinite values" refers to values larger than normal finite representation, including both exploded values and infinity.

[0015] "Negligible values" refers to values that are too small to be represented in normal form, including both vanished and zero values. These values have magnitudes that are treated as insignificant in relative arithmetic operations.

[0016] "Undefined state" refers to values resulting from mathematically undefined operations (such as $0 / 0$ or $\infty - 42$). Unlike traditional floating-point formats with a single NaN representation, this invention provides multiple undefined states with specific bit patterns that preserve the cause of the ambiguity.

[0017] "Absolute operation" refers to an arithmetic operation where sign handling follows fixed rules regardless of the scale of operands, such as multiplication and division. These operations work consistently with normal, exploded, vanished, zero and infinite values.

[0018] "Relative operation" refers to an arithmetic operation where sign handling depends on the signs and magnitudes of operands, such as addition and subtraction.

[0019] "Overflow" refers to when a calculation produces a result whose magnitude exceeds the maximum representable range. In this invention, overflow transitions a value to an exploded state rather than saturating to infinity, preserving phase/orientation information and allowing certain operations to continue.

[0020] "Underflow" refers to when a calculation produces a result whose magnitude is below the minimum representable range. In this invention, underflow transitions values to a vanished state rather than truncating to zero.

[0021] "Integer" as used in type parameters refers to a signed integer type of any bit width. While typical implementations use standard integer sizes (8, 16, 32, 64, or 128 bits), the invention can utilize any bit width for both fraction and exponent components.

[0022] "Real" refers to the component of a number that lies on the conventional number line. In this invention, real values can be represented as standalone Scalar types or as the real component of a Circle type. Within a Circle, the real component stores the coefficient of the real part of a complex number. The real component shares an exponent with the imaginary component to maintain alignment between both fractions thus greatly simplifying complex operations.

[0023] "Imaginary" refers to the component of a complex number that represents multiples of the imaginary unit i , where $i \times i = -1$.

[0024] "Infinity" refers to a specific representation of mathematical infinity as a singularity within the numeric space. Unlike IEEE-754's approach with separate positive and negative infinities, this invention treats infinity as a unified concept.

[0025] "Zero" refers to the representation of mathematical zero as a singularity within the numeric space. Unlike IEEE-754's approach with separate positive and negative zeros, this invention treats zero as a unified concept at the middle of the continuous number line.

BRIEF DESCRIPTION OF THE FIGURES

[0026] FIG. 1 illustrates the fundamental discontinuity in IEEE-754's number representation, showing how the sign-magnitude approach creates a break at zero with overlapping positive and negative values that only differ by the sign bit.

[0027] FIG. 2A-2B contrasts the invention's two's complement representation with IEEE-754, demonstrating how the invention achieves a continuous number line where values maintain mathematical relationships across the entire range. The figure shows how zero appears naturally in the center rather than as a special case.

[0028] FIG. 3 illustrates the Scalar type implementation, showing the parameterized structure with fraction and exponent components that can be configured with various integer sizes. The fraction component stores the normalized significand (called phase), while the exponent determines the scale of the phase.

[0029] FIG. 4 presents the Circle type definition for complex numbers, showing how real and imaginary phases share a common exponent for efficient complex arithmetic operations.

[0030] FIG. 5 demonstrates the type flexibility of the invention, showing how any integer size can be used for both fraction and exponent components, allowing applications to optimize for precision and range according to their specific requirements.

[0031] FIG. 6 presents the precision-range matrix of FE type combinations, illustrating the flexibility of the system to accommodate different application needs from limited precision with wide range to high precision with narrower range.

[0032] FIG. 7 visually illustrates the different normalization levels used to represent various numeric states. The figure shows how specific bit patterns in the most significant positions encode normal values, escaped values (exploded and vanished), and different categories of undefined states.

[0033] FIG. 8 provides an excerpt of the undefined state definitions, showing how different bit patterns are used to preserve the cause of ambiguous operations, enabling more effective debugging and error tracing compared to IEEE-754's single NaN.

[0034] FIG. 9 presents the Addition Operation Truth Table, illustrating how different value types interact under addition in the SPIRIX system. The table demonstrates the mathematical consistency of addition across normal, vanished, exploded, and special values.

[0035] FIG. 10 presents the Subtraction Operation Truth Table, showing outcomes of subtracting different value types in the SPIRIX system. The minuend (value being subtracted from) appears along the top, and the subtrahend (value being subtracted) appears along the left column, with a note about negation range limits.

[0036] FIG. 11 presents the Multiplication Operation Truth Table, demonstrating the system's multiplicative consistency. The table shows how the product of non-zero values is never zero, preserving phase information even when results exceed normal representation range.

[0037] FIG. 12 presents the Division Operation Truth Table, illustrating division behavior across all value types. The table shows how division by zero consistently produces infinity for non-zero numerators and how division operations preserve mathematical relationships between vanished and exploded values.

[0038] FIG. 13 illustrates the SPIRIX number line and comparison ordering, showing how values are arranged from positive exploded at the top to negative exploded at the bottom, with zero as a single point in the middle. The figure demonstrates how infinity and undefined states are treated as incomparable.

[0039] FIG. 14 presents a flowchart of the IEEE-754 subtraction operation, illustrating the multiple branching paths required for handling different sign combinations, rounding, and denormalized values.

[0040] FIG. 15 presents a flowchart of the scalar subtraction algorithm, demonstrating how normal and ambiguous values are differentiated with one check in the main path. Underflow is checked and handled before returning. Overflow is naturally handled by the system design.

[0041] FIG. 16 contains an annotated assembly instruction trace for normal-normal IEEE-754 Binary32 <f32> subtraction, illustrating the branching control flow and numerous special case tests and steps required for handling different sign combinations, rounding, and special values.

[0042] FIG. 17 shows annotated assembly instructions for scalar subtraction, highlighting how two's complement arithmetic naturally handles sign during operations, eliminating the need for sign bit testing and greatly reducing special case handling required by IEEE-754.

[0043] Appendix A provides the complete assembly instruction trace for a normal complex division between two IEEE-754 Binary32 <f32> complex numbers, illustrating the substantially greater complexity (approximately 2,199 instructions) required to handle just a normal/normal case in the IEEE approach. Analysis of the IEEE trace reveals 165 conditional jumps, 82 function calls, and 83 return instructions, creating a branching maze that impacts performance and predictability.

[0044] FIG. 18 provides assembly instructions for complex number division using the F5E4 Circle type (32-bit fractions, 16-bit exponent). This implementation demonstrates how normal complex division is handled in 63 instructions. Complete handling of all cases is illustrated in 169 instructions.

[0045] FIG. 19 demonstrates a mathematically meaningful bitwise AND operation between two Scalar values, showing how the system aligns operands by exponent before applying bitwise logic, preserving numerical significance thruout the operation.

[0046] FIG. 20 illustrates how bitwise operations respect two's complement properties when operating on signed values, showing examples of AND, OR, XOR, and NOT operations with positive and negative operands. This demonstration highlights how sign information naturally propagates thru bitwise operations.

[0047] FIG. 21 presents the implementation of bit shift operations as checked exponent adjustments rather than direct bit manipulation. This approach preserves the mathematical interpretation of shifts as multiplication or division by powers of two.

DETAILED DESCRIPTION

[0048] The invention implements a two's complement floating-point arithmetic system with distinct configurations for real and complex numbers. Real numbers are represented by the "Scalar" type, while complex numbers are represented by the "Circle" type. Both configurations are parameterized by fraction (F) and exponent (E) bit widths, allowing customization based on application range and precision requirements. Implementations are given in Rust and Intel x86-64 bit assembly language, however these can be implemented in any integer mathematics processor.

IEEE-754 vs. Two's Complement Representation

[0049] As illustrated in FIG. 1, the IEEE-754 sign-magnitude representation creates a fundamental discontinuity at zero. This visualization reveals how every value has an opposite representation—positive and negative—that differ only by the sign bit, creating a mirror of the entire number line. This includes cases like zero, which exists as both +0 and -0, despite attempting to represent the same mathematical value. This sign-magnitude approach introduces significant branching complexity in numerical operations, requiring separate code paths for different sign combinations and explicit testing of the sign for most arithmetic operations.

[0050] In contrast, FIG. 2A presents a linear view of the invention's continuous two's complement representation. Unlike IEEE-754, the invention achieves a completely continuous number space where values wrap around to maintain mathematical relationships across the entire range, eliminating the discontinuity at zero. This continuity enables operations to cross the zero boundary without any special handling or branching.

[0051] FIG. 2B further illustrates this concept with a vertically rotated perspective of the same representation, mapping values from fraction $\text{MAX}/2+1$ at the bottom to fraction $\text{MAX}/2$ at the top, with zero naturally landing in the middle. This visualization highlights how zero becomes a contiguous value in the number line rather than a special case requiring separate handling.

Core Configuration Definitions

[0052] The Scalar and Circle configurations form the core of the system. These definitions provide a unified framework for representing and operating on both real and complex numbers.

[0053] As shown in FIG. 3, the Scalar type implements the representation for real numbers thru two primary components: a normalized two's complement fraction that determines precision, and an unbiased two's complement exponent that determines range. The parameterized design allows for independent selection of fraction and exponent sizes to match application requirements.

[0054] FIG. 4 presents the Circle type implementation for complex numbers. The structure contains real and imaginary fractions sharing a common exponent, enabling efficient complex arithmetic operations while maintaining mathematical consistency. By using a shared exponent for both fractions, the Circle type maintains alignment between real and imaginary components, greatly simplifying complex arithmetic operations compared to traditional approaches that handle components independently.

[0055] The type flexibility of the invention is demonstrated in FIG. 5, showing how any integer size can be used for both fraction and exponent components. This abstraction over integer types allows the system to adapt to different precision and range requirements. While the implementation shown uses standard signed integer sizes (i8 thru i128), the system can work with arbitrary-sized integers, allowing for precise tuning of numeric representations to application-specific needs.

[0056] FIG. 6 illustrates the precision-range matrix of valid FE type combinations. This table demonstrates how the invention enables independent configuration of precision (controlled by fraction size) and range (controlled by exponent size). The precision values shown are calculated as the number of decimal digits representable with the given fraction bits, accounting for the sign bit. For ranges exceeding standard scientific notation, the table uses nested exponential notation ($10^{(10^x)}$) to represent the scale of normal values for each exponent size.

[0057] Scalar values are calculated as:

[0058] $value = fraction \times 2^{exponent}$

[0059] Circle values have real and imaginary phases sharing a common exponent:

[0060] $value = (real + imaginary \times i) \times 2^{exponent}$

Normalization Levels and Numerical States

[0061] The invention employs a leading same bit normalization scheme to represent different numerical states thru specific bit patterns in the fraction component(s).

[0062] As depicted in FIG. 7, the normalization levels (N-levels) encode different numerical states thru distinct bit patterns in the most significant positions of the fraction. Normal values use N-1 normalization with bit patterns 01... or 10... in the most significant positions, while vanished values use N-2 normalization, and various undefined states are represented thru higher normalization levels.

[0063] A key innovation in the normalization scheme is the concept of "escaped" values—those that exceed normal representation range, but their phase is known. Unlike IEEE-754, which uses special values like infinity or NaN that lose information about the operation, the invention's escaped values preserve phase/orientation information, enabling continued participation in absolute operations with magnitudes outside normal range. For example a negative vanished Scalar multiplied by another negative vanished Scalar will never return zero ($[-\downarrow] \times [-\downarrow] \neq [0]$) and the result will be a positive vanished Scalar ($[-\downarrow] \times [-\downarrow] \rightarrow [+ \downarrow]$), thus preserving orientation and continuity from zero to infinity wherever possible.

Unified Treatment of Zero [0], Infinity [∞], and Escaped Values

[0064] The invention implements a mathematically consistent approach to the singularities of zero and infinity while acknowledging fundamental computational limitations. Unlike IEEE-754, which treats these values as both positive and negative.

[0065] Zero is represented as a specific bit pattern of all zeroes in the fraction, paired with an ambiguous exponent. SPIRIX treats zero as a singular point on the number line rather than as two distinct values [0].

[0066] Similarly, infinity is represented as a specific bit pattern of all ones in the fraction component, also with an ambiguous exponent [∞]. The system maintains mathematical relationships involving infinity, such as:

[0067] - Any non-zero value divided by zero produces infinity

[0068] - Infinity multiplied by any non-zero value remains infinity

[0069] - Zero divided by any non-zero value remains zero

[0070] - Adding or subtracting transfinite values produces a distinct undefined state

[0071] While the system strives to follow Riemann principles, practical computation with finite representation necessarily departs from mathematical abstraction. When values escape normal representation, we preserve WHICH direction they approach zero or infinity (their phase/orientation), but not HOW far they are in that direction. This creates unavoidable ambiguities where addition and subtraction operations break with escaped values because they depend on known magnitudes.

[0072] The invention addresses these limitations by explicitly marking ambiguous operations with distinct bit patterns that preserve the operation cause. This approach delivers significant advantages over IEEE-754, which silently truncates operations like MIN_POSITIVE^2 to zero, violating multiplicative zero equality. By acknowledging computational limits while preserving phase information, the invention enables more reliable numerical algorithms and improved error tracing.

[0073] By treating both zero and infinity as orientationless singular points, while tracking the phase of values that escape normal representation, the invention enables consistent behavior across operations, greatly reducing special case handling while preserving mathematical coherence beyond representable boundaries wherever possible.

Undefined States and Error Preservation

[0074] One of the most significant innovations of the invention is its approach to undefined numerical states, as illustrated in FIG. 8. Unlike IEEE-754's single NaN (Not a Number) representation, which collapses all error conditions into a single state, the system employs distinct bit patterns to encode the specific cause of undefined operations. The figure demonstrates how different operations that produce undefined results (such as $0/0$, $\infty+1$, or 0^0) are assigned unique bit patterns in the fraction component, always with an ambiguous exponent.

[0075] This preservation of error cause enables error tracing in numerical applications. When an undefined result propagates thru subsequent operations, the system maintains the original cause rather than losing this critical information, allowing developers to trace computational issues to their source. The hierarchical organization of undefined states further enhances this capability, with different operations (basic arithmetic, powers, logarithms, trigonometric functions) assigned different ranges within the bit pattern space. Each of these patterns, given sufficient fraction size can provide other important undefined information or custom implemented behavior.

SPIRIX Operation Truth Tables

[0076] The invention attempts to implement consistent and helpful behavior across numerical operations and states. The SPIRIX Circle and Scalar states follow Riemann principles when performing operations.

[0077] FIG. 9 above presents the Addition Operation Truth Table, illustrating how different states interact under addition in the SPIRIX system. The table demonstrates the mathematical consistency of addition across normal, vanished, exploded, and special states. Note that operations between certain states, such as adding an exploded and a normal, produce specific undefined states, preserving information about the operation that led to the undefined state.

[0078] The Subtraction Operation Truth Table in FIG. 10 shows the outcomes of subtracting different states in the SPIRIX system. The minuend (value being subtracted from) appears along the top, and the subtrahend (value being subtracted) appears along the left column. The table highlights how subtraction maintains mathematical coherence even in boundary cases. Note the minimum negative and minimum positive explode or vanish upon negation of the subtrahend.

[0079] FIG. 11 demonstrates the system's multiplicative consistency thru the Multiplication Operation Truth Table. A key feature illustrated in this table is that the product of two non-zero states is never a zero state, preserving a fundamental mathematical property that can be violated in IEEE-754 implementations due to multiplicative underflow to zero or overflow to infinity. This multiplicative identity property is expressed as $a \times b = 0$ if and only if $a = 0$ or $b = 0$. The table also shows how the system maintains phase information even when results exceed normal range, transitioning to exploded or vanished states rather than truncating to infinities or zeros.

[0080] The multiplicative identity is preserved because SPIRIX treats vanished values as distinct from zero. When a value becomes too small for normal representation, rather than truncating it to zero (as in IEEE-754), SPIRIX transitions it to a vanished state. Therefore, when a normal or vanished value is multiplied by a vanished value, the result remains vanished with the appropriate phase, never collapsing to zero. Similarly, overflow operations that would produce infinity in IEEE-754 instead produce exploded states in SPIRIX, again maintaining phase information and preserving the property that products of finite values remain finite.

[0081] The Division Operation Truth Table in FIG. 12 illustrates division behavior for all Circle and Scalar states. The table shows how division by zero produces infinity for non-zero numerators, while division of zero by any non-zero value produces zero. This implements the mathematical principle that division of zero produces zero, and division by zero produces infinity when the numerator is non-zero. The singular case of zero divided by zero remains properly undefined rather than producing a quiet or signaling NaN as in IEEE-754.

[0082] Division operations between escaped values (exploded and vanished) follow Riemann principles, with vanished divided by exploded producing vanished, and exploded divided by vanished producing exploded. This behavior eliminates many special cases that would otherwise require explicit handling in numerical applications.

[0083] The system implements special cases for operations involving zero and infinity. The rules enforce that $0 \times \infty$ produces an undefined result with a specific bit pattern indicating the cause. Similarly, $\infty + \infty$ and $\infty - \infty$ produce distinct undefined results. These undefined values provide error cause tracing that is absent in IEEE-754's collapsed NaN representation, enabling more effective debugging of numerical algorithm issues.

[0084] FIG. 13 presents the SPIRIX Scalar number line and comparison ordering, illustrating the continuous nature of the two's complement representation. This figure illustrates how normal, vanished, and exploded values maintain a consistent ordering relationship with a singular zero, while infinity and undefined states remain incomparable.

IEEE-754 vs. SPIRIX Implementation Complexity

[0085] The fundamental differences in representation approach between IEEE-754 and the invention result in significant implementation differences, particularly in terms of complexity and instruction count.

[0086] FIG. 14 illustrates the control flow for IEEE-754 subtraction. The flowchart reveals numerous branching paths needed to handle different sign combinations, exponent comparisons, and special value considerations.

[0087] In contrast, FIG. 15 demonstrates the control flow for Scalar subtraction. The diagram shows two primary paths: one for normal values and one for ambiguous values, with the normal path aligning and subtracting fractions, then checking and updating the exponent without denormal, sign, exponent bias or implied one steps. Note how addition and subtraction only need to check for underflow. Overflow is naturally handled by the placement of the ambiguous exponent where the largest magnitude wraps around to our ambiguous value.

[0088] The assembly instruction trace for IEEE-754 Binary32 subtraction in FIG. 16 further highlights the implementation complexity of IEEE-754 floating-point operations. The trace reveals approximately 192 machine instructions with complicated branching even for this normal case. This trace has 7 function calls, 6 unconditional jumps, 10 conditional jumps, and 7 returns, creating potential pipeline stalls and prediction failures in modern processors.

[0089] FIG. 17 presents the assembly implementation of Scalar subtraction using the invention's two's complement method. The assembly instructions demonstrate how most normal cases can be completed in 33 instructions. All normal Scalar subtraction cases resulting in normal, vanished, exploded or zero states are handled in 49 total instructions, with most normal cases completing without any jumps.

Appendix A: IEEE-754 Complex Division ASM Trace

[0090] **Appendix A** provides the full assembly instruction trace for a normal complex number division operation returning a normal complex number using the IEEE-754 Binary32 format. This trace illustrates the complexity of complex arithmetic in conventional floating-point systems. The trace captures each instruction executed during the complex division operation. Note how components are not aligned, thus requiring individual component operations to perform the operation.

[0091] The instruction trace in **Appendix A** consists of 2,199 total assembly instructions for division with a normal numerator and normal denominator resulting in a normal quotient. Of these instructions, 165 are conditional jumps that create decision points in the execution path, 82 are function calls to various subroutines, and 83 are return instructions.

[0092] This implementation complexity stems from several architectural characteristics of IEEE-754. First, the sign-magnitude representation necessitates explicit sign bit tests and different execution paths for each sign combination. Second, each component of a complex number (real and imaginary parts) must be processed independently, without any shared representation or processing mechanism. Third, special values such as signed zeros and signed infinities require explicit detection and dedicated handling routines. These characteristics combine to create a multiplicative effect on instruction count when implementing operations like complex division.

SPIRIX Circle Complex Division ASM implementation

[0093] The invention's approach to complex arithmetic represents an advancement over traditional methods, particularly in terms of implementation efficiency.

[0094] FIG. 18 presents the complete assembly implementation for complex number division for all cases using the F5E4 Circle type (32-bit fractions, 16-bit exponent). This implementation demonstrates how normal floating-point complex division can be handled in 63 instructions, while handling of all cases requires 169 instructions.

[0095] The shared exponent aligns the real and imaginary components to maintain relative scaling thruout operations, simplifying implementation while preserving mathematical consistency. This approach is particularly beneficial for applications involving extensive complex arithmetic, such as signal processing, electrical engineering, and quantum computing simulations.

Bitwise Operations on Floating-Point Values

[0096] A unique innovation in the invention is its implementation of mathematically meaningful bitwise operations directly on floating-point values, a capability completely absent in traditional floating-point standards.

[0097] As illustrated in FIG. 19, the invention enables direct bitwise operations on Scalar values while preserving numerical meaning. The example demonstrates a bitwise AND operation between two Scalar values, showing how the system first aligns the binary representations based on their exponents, then performs the bitwise operation on the aligned values. The result maintains a mathematically meaningful relationship to the inputs, unlike traditional implementations where bitwise operations on floating-point numbers are implemented thru type conversion to integers and back.

[0098] FIG. 20 demonstrates how bitwise operations in the invention respect two's complement properties when handling signed values. The examples of AND, OR, XOR, and NOT operations with positive and negative operands highlight how sign information naturally propagates thru bitwise operations. This consistent behavior extends across normal values, escaped values (exploded and vanished), and all other numeric states within the system, maintaining mathematical coherence wherever possible.

[0099] The implementation of bit shift operations as checked exponent adjustments, shown in FIG. 21, further illustrates the invention's mathematically consistent approach to bitwise operations. Rather than directly manipulating values, shift operations are implemented as exponent adjustments, preserving the mathematical interpretation of shifts as multiplication or division by powers of two. This approach ensures that shifts maintain mathematical coherence, including with escaped values, while eliminating the complexity and potential for errors that can occur with direct bit manipulation.

Subtractive and Multiplicative Consistency Without Denormals

[0100] A significant innovation in the invention is how it maintains mathematical consistency and subtractive equality properties without requiring denormal numbers. In IEEE-754, denormal numbers were primarily introduced to address a specific edge case: maintaining the mathematical property that $x = y$ if and only if $x - y = 0$. Without denormals, IEEE-754 encounters situations where two distinct representable numbers subtract to produce zero because their difference would be too small to represent as a normalized number and truncated to zero, potentially causing unexpected behavior in algorithms that rely on this mathematical property.

[0101] The invention's two's complement representation with the "vanished" state solves this problem using a different mechanism. When a value becomes too small for normal representation, it transitions to a vanished state. Unlike IEEE-754 denormals, which represent tiny values with reduced precision, the invention's vanished values represent a categorical state of "too small to represent normally." This approach maintains the critical property that equal values always have a zero difference, while eliminating the implementation complexity and performance penalties associated with denormal handling.

[0102] Consider two values a and b that are very close but not identical. Their difference $a - b$ will either be a normal value (if the difference is representable) or a vanished value (if the difference is too small for normal representation). In either case, the difference will be non-zero, preserving the property that $a = b$ if and only if $a - b = 0$. Significantly, this holds true without requiring the extensive special handling that denormals introduce.

[0103] The invention also improves mathematical consistency in multiplicative and reciprocal operations beyond normal ranges. Unlike IEEE-754, which allows products of non-zero values to become zero when they underflow, the invention guarantees that the product of any two non-zero values always produces a non-zero result, preserving fundamental mathematical principles. Similarly, reciprocal operations maintain mathematical coherence: *nonzero* / 0 produces an infinite result with a specific bit pattern, while 1 / *vanished* produces an exploded result with the appropriate sign.

Simplified Rounding

[0104] The invention dramatically simplifies rounding compared to IEEE-754 due to its continuous number line and consistent two's complement representation. The system's use of double-wide fractions during intermediate calculations provides ample residual bits for precise rounding decisions.

[0105] All standard rounding modes become straightforward to implement: Round down (truncation) simply discards residual bits after calculation. Round up adds 1 to the lowest retained bit if any residual bits are non-zero, working naturally for both positive and negative numbers due to the continuous nature of the number line. Round to nearest adds 1 to the lowest retained bit if the high residual bit is a 1 and ties to even utilizes the LSB. Rounding, which requires sign-dependent logic in IEEE-754, becomes a straightforward sign agnostic operation in SPIRIX without needing to mathematically flip the entire number line based on the sign of the value.

[0106] Unlike IEEE-754, which requires complex logic for handling sign bits and special values during rounding, SPIRIX's continuous number line ensures rounding maintains mathematical consistency across the numeric spectrum, including the zero boundary. The contiguous representation from minimum to maximum eliminates separate rounding paths based on sign, significantly reducing implementation complexity and making corner cases like signed zeros unnecessary.

[0107] With SPIRIX's continuous representation, rounding simply involves examining residual bits and potentially making a single adjustment to the fraction.

TECHNICAL ADVANTAGES

[0108] The invention provides several significant technical advantages over traditional floating-point implementations:

[0109] - Simplified computational architecture: The elimination of separate sign bit handling and extensive special case branching results in faster, smaller and more efficient floating point hardware implementations. The architectural simplification reduces the complexity of branch prediction logic and speculative execution mechanisms in processor pipelines.

[0110] - Enhanced power efficiency: The substantial reduction in conditional branches and calculation pipeline stages significantly decreases power consumption per operation. This technical improvement enables more energy-efficient computation, particularly beneficial in performance critical environments such as mobile devices, embedded systems, and high-density computing clusters.

[0111] - Instruction path optimization: The dramatic reduction in instruction count for numerical operations (demonstrated by the 34:1 ratio improvement for complex division) represents a significant advancement in computational efficiency. This technical enhancement is particularly evident in instruction-limited domains such as scientific computing, signal processing, and machine learning, where it enables complex algorithms to execute within available computational constraints.

[0112] - Error state preservation: The system's approach to undefined states provides tracing of error conditions thru preserved operation causes, enabling more effective error isolation and debugging. This technical capability eliminates the information loss inherent in IEEE-754's single NaN representation, allowing for automated identification of specific numerical failure modes.

[0113] - Parameterized numerical representation: The invention's parameterized design with independent fraction and exponent sizing provides a flexible framework that can be precisely tuned to application requirements. This allows for technical optimization across different use cases without requiring separate implementations, enabling a single codebase to adapt to diverse precision and range requirements.

[0114] - Mathematical identity preservation: The invention's approach to representing singularities such as zero and infinity preserves fundamental mathematical properties that IEEE-754 compromises. While IEEE-754 maintains the identity that $a-b=0$ if and only if $a=b$ thru denormal numbers, this invention achieves the same mathematical property without the complex denormalization mechanisms that impact performance. More significantly, the system preserves critical multiplicative and reciprocal identities by ensuring that non-zero values multiplied together never produce zero (underflow produces vanished states that maintain phase information), finite values multiplied together never inappropriately produce infinity, and operations maintain consistent behavior with zero and infinity. This mathematical coherence represents a technical advancement that enables more reliable numerical algorithms with fewer computational artifacts and more predictable behavior across the entire numeric space.

IMPLEMENTATION EMBODIMENTS

[0115] The invention described herein can be realized across diverse computational platforms and embodiments, as the core principles of two's complement floating-point arithmetic transcend specific implementation technologies. While the examples provided use Rust programming language and x86-64 assembly language for illustration, the invention is fundamentally language-agnostic and architecture-independent.

[0116] Software embodiments can leverage standard integer operations available in virtually any programming language or computing environment. Library implementations may employ generics or templates to parameterize the fraction and exponent sizes, enabling application-specific tuning of precision and range characteristics. The following implementation approaches are possible:

[0117] - General-purpose processor implementations can leverage native integer operations and SIMD (Single Instruction, Multiple Data) parallelism. The consistent branching patterns and uniform operations across all numerical states make the invention particularly well-suited for modern superscalar processors with branch prediction capabilities.

[0118] - Hardware implementations can achieve significant efficiency improvements thru specialized circuit designs. These may include:

- Simplified arithmetic logic units (ALUs) that eliminate sign-handling circuitry
- Dedicated normalization components for left aligned two's complement representations
- Specialized complex arithmetic accelerators that capitalize on the shared exponent

[0119] - FPGA implementations can realize the invention thru configurable logic blocks that directly implement the binary operations described herein. The reduced branching complexity translates to simpler state machines and more efficient resource utilization compared to traditional floating point representations.

[0120] - GPU implementations can exploit the parallel architecture to perform high-thruput floating-point operations. The uniform computational flow across different numerical states aligns well with the SIMT (Single Instruction, Multiple Thread) execution model of modern graphics processors.

[0121] The reference implementation provided in assembly language and Rust demonstrates the practical realization of these concepts, but should be understood as merely exemplary rather than limiting. Any computational device capable of integer arithmetic can implement this invention, from embedded microcontrollers to high-performance computing clusters, with appropriate adaptations to the specific constraints and capabilities of the target platform.

[0122] Future implementations might incorporate domain-specific extensions such as:

- Specialized transcendental function units optimized for two's complement
- Adaptive precision mechanisms that dynamically adjust fraction and exponent sizes
- Hybrid number systems that combine aspects of this invention with other approaches to floating-point representation

CONCLUSION

[0123] The invention represents a significant advancement in floating-point arithmetic, addressing many of the longstanding limitations of IEEE-754 while providing new capabilities for numerical computation. By leveraging the natural properties of two's complement arithmetic and applying them consistently throughout the calculation pipeline, the invention achieves simplified implementation, error tracking, mathematical continuity, configurable precision, and unified complex arithmetic.

[0124] These advantages make the invention particularly well-suited for applications in scientific computing, machine learning, signal processing, graphics, and other domains where floating point computation forms a core component. The described technology enables both improved performance and reduced development complexity for numerical integration across a wide range of computing platforms.

[0125] The modular design and mathematical foundations provide a solid basis for future extensions and adaptations while maintaining backward compatibility with existing implementations. As computing continues to evolve toward more specialized architectures and domain-specific accelerators, the principles described in this invention offer a solid framework for consistent, efficient numerical representation across heterogeneous computing environments.

REFERENCES

- [1] IEEE 754-1985. "IEEE Standard for Binary Floating-Point Arithmetic." Institute of Electrical and Electronics Engineers, 1985.
- [2] IEEE 754-2019. "IEEE Standard for Floating-Point Arithmetic." Institute of Electrical and Electronics Engineers, 2019.
- [3] Goldberg, David. "What Every Computer Scientist Should Know About Floating-Point Arithmetic." ACM Computing Surveys, Vol. 23, No. 1, March 1991.
- [4] Kahan, William. "IEEE Standard 754 for Binary Floating-Point Arithmetic." Lecture Notes on the Status of IEEE Standard 754, 1997.
- [5] Gustafson, John L. "The End of Error: Unum Computing." Taylor & Francis, 2015.
- [6] Gustafson, John L. "Beating Floating Point at its Own Game: Posit Arithmetic." South Ural State University, 2017.

CLAIMS

1. A system for representing and processing numerical values, comprising:
 - (a) a processor; and
 - (b) a memory storing instructions which, when executed by the processor, implement a floating-point arithmetic system that:
 - i. represents floating-point values using two's complement fractions and unbiased two's complement exponents thruout the entire pipeline;
 - ii. utilizes a single reserved exponent value to identify values outside normal representation, including zero, infinity, and undefined values;
 - iii. employs multiple normalization levels encoded in the most significant bits of the fraction component, including:
 - A. N-1 normalization with patterns 01... or 10... in most significant bits for normal and exploded values, preserving sign information;
 - B. N-2 normalization with patterns 001... or 110... in most significant bits for vanished values;
 - C. specific bit patterns for zero, infinity, and undefined states; and
 - iv. performs arithmetic operations by directly operating on the two's complement fractions while maintaining proper alignment thru exponent adjustment, without requiring separate sign bit testing or handling.
2. The system of claim 1, wherein the system implements a continuous numerical representation by:
 - (a) utilizing two's complement representation to create a continuous number line with no discontinuity at zero;
 - (b) transitioning values below minimum representable magnitude to vanished states with N-2 normalization rather than truncating to zero, thereby maintaining that $a - b = 0$ if and only if $a = b$ without requiring denormalized fractions;
 - (c) transitioning values above maximum representable magnitude to exploded states with N-1 normalization rather than saturating to infinity; and
 - (d) aligning fractions by exponent adjustment before performing arithmetic operations to maintain consistent behavior across the entire range.

3. The system of claim 1, wherein the system preserves multiplicative and reciprocal identity by:
 - (a) representing zero with a specific bit pattern (all zeros in fraction) distinct from vanished values;
 - (b) performing multiplication or division by aligning and multiplying or dividing two's complement fractions while adding or subtracting their exponents;
 - (c) transitioning values below minimum representable magnitude to vanished states with N-2 normalization rather than truncating to zero, thereby ensuring that the product or quotient of non-zero or non-infinite values never equals zero or infinity; and
 - (d) transitioning values above maximum representable magnitude to exploded states with N-1 normalization rather than saturating to infinity, thereby ensuring that products or quotients of finite values maintain sign/orientation information.
4. The system of claim 1, wherein the system encodes the sign/orientation of a value within the fraction's two's complement representation by:
 - (a) normalizing to the N-1 level with patterns 01... for positive values and 10... for negative values in the most significant bits, where the first bit indicates sign and the second bit is the MSB of the fraction value;
 - (b) for complex numbers, using the component (real or imaginary) with larger magnitude to determine the Leading Sign Bits (LSB) pattern for normalization;
 - (c) performing arithmetic operations directly on these normalized fractions without requiring separate sign handling; and
 - (d) maintaining sign/orientation information thruout the calculation pipeline utilizing the inherent properties of two's complement.

5. A method for processing floating-point values in a computing system, comprising:
 - (a) representing floating-point values using two's complement fractions and unbiased two's complement exponents;
 - (b) normalizing fraction components to specific bit patterns that encode numerical states, including:
 - i. using N-1 normalization (patterns 01... or 10... in most significant bits) for normal values;
 - ii. using N-2 normalization (patterns 001... or 110... in most significant bits) for vanished values;
 - iii. using all zeros pattern for zero and all ones pattern for infinity;
 - iv. using a single reserved exponent value to indicate values outside normal representation;
 - (c) aligning fraction components by exponent adjustment before performing arithmetic operations to maintain binary point alignment; and
 - (d) transitioning values outside representable range to exploded or vanished states rather than truncating to infinity or zero, thereby preserving phase information.
6. The method of claim 5, wherein the two's complement representation creates a continuous number line across zero without redundant zero values, thereby enabling:
 - (a) unified processing paths for operations regardless of sign;
 - (b) elimination of explicit sign bit testing and adjustment; and
 - (c) consistent handling of operations that cross zero.

7. The method of claim 5, wherein numerical operations require fewer conditional branches than equivalent operations using separate sign bit representation, as demonstrated by:
 - (a) performing scalar subtraction in approximately 33 instructions for normal cases compared to approximately 200 instructions in IEEE-754 implementations;
 - (b) implementing jumpless complex division in exactly 63 instructions for normal cases compared to 165 conditional jumps and 2,199 instructions in IEEE-754 implementations; and
 - (c) reducing conditional jumps from more than 100 to fewer than 10 for average complex arithmetic operations.
8. A system for complex number processing, comprising:
 - (a) a processor; and
 - (b) a memory storing instructions which, when executed by the processor, implement:
 - i. a complex number representation that uses:
 - A. real and imaginary components as two's complement fractions;
 - B. a shared exponent that ensures both components maintain relative magnitudes;
 - C. specific bit patterns in most significant positions of fractions to encode infinite, zero, normal, exploded, vanished, and undefined states;
 - ii. arithmetic operations that maintain mathematical consistency across the entire complex plane, including:
 - A. performing complex division with 1 reciprocal and 6 multiplications;
 - B. encoding complex infinity thru a specific bit pattern;
 - C. preserving angular information when complex values exceed normal representation range;
 - iii. a unified normalization process that simultaneously adjusts both real and imaginary components based on the highest normalization level in either component.

9. The system of claim 8, wherein the complex number representation implements efficient complex arithmetic operations by:
 - (a) using a shared exponent to align real and imaginary components, eliminating the need for separate normalization and exponent of each component;
 - (b) applying normalization shifts to both real and imaginary components simultaneously; and
 - (c) providing distinct remainder operations: mathematical modulus based on complex division semantics and component-wise modulo that performs independent modulo on each component.
10. A method for identifying causes of undefined numerical results in a floating-point arithmetic system, comprising:
 - (a) encoding different categories of undefined operations as distinct bit patterns in a fraction component, wherein the bit patterns are organized hierarchically by normalization level, including:
 - i. N-3 normalization patterns for basic arithmetic undefined operations;
 - ii. N-4 normalization patterns for power and root undefined operations;
 - iii. N-5 normalization patterns for logarithm undefined operations;
 - iv. N-6 normalization patterns for trigonometric undefined operations;
 - (b) utilizing a single reserved exponent value to indicate all values outside normal representation, including undefined states; and
 - (c) preserving the original undefined bit pattern thru subsequent operations, thereby maintaining information about the first operation that caused the undefined result.

11. An arithmetic system with a unified mechanism for handling special values, comprising:

(a) a processor; and

(b) a memory storing instructions which, when executed by the processor, implement:

i. a representation where zero and infinity are singular points with specific bit patterns:

A. Zero represented by all zeros in the fraction with the reserved exponent value;

B. Infinity represented by all ones in the fraction with the reserved exponent value;

ii. operations that maintain fundamental mathematical identities for these special values:

A. non-zero value divided by Zero produces Infinity;

B. Zero divided by non-zero value produces Zero;

C. non-zero value multiplied by Infinity produces Infinity;

D. Infinity multiplied by Zero produces an undefined state with a specific bit pattern;

iii. specific undefined states for indeterminate forms like Zero/Zero and Infinity-Infinity, each with distinct bit patterns that preserve the cause.

12. A system for processing numerical values with configurable precision and range, comprising:

(a) a processor; and

(b) a memory storing instructions which, when executed by the processor, implement:

i. a numeric format wherein fraction and exponent field sizes are independently configurable using integer types, allowing selection of:

A. precision thru fraction size corresponding to different integer bit widths; and

B. range thru exponent size corresponding to different integer bit widths;

ii. a consistent arithmetic processing pipeline that maintains normalization and state detection across all precision and range configurations; and

iii. a mechanism for selecting optimal fraction/exponent combinations based on application precision and range requirements.

13. A method for implementing bitwise operations directly on floating-point values in a two's complement floating-point arithmetic system, comprising:
- (a) aligning operands by exponent adjustment to ensure bits of equivalent mathematical significance are paired, including:
 - i. expanding fractions to double-wide representation;
 - ii. shifting the fraction with the smaller exponent by the exponent difference;
 - iii. maintaining two's complement sign properties during alignment;
 - (b) applying bitwise logic operations to the aligned fractions while preserving two's complement properties, thereby maintaining proper sign propagation; and
 - (c) implementing bit shift operations as exponent adjustments rather than direct bit manipulation:
 - i. implementing left shift as an increase to the exponent by the shift amount;
 - ii. implementing right shift as a decrease to the exponent by the shift amount;
 - iii. transitioning to exploded or vanished states when shift operations would exceed the representable range.
14. The method of claim 13, wherein aligning operands by exponent ensures that the bitwise operations produce mathematically meaningful results by:
- (a) ensuring corresponding bits represent values of equivalent mathematical significance before applying bitwise operations; and
 - (b) maintaining proper two's complement semantics for both positive and negative values during bitwise operations.

15. A method for rounding floating-point values in a computing system, comprising:
- (a) representing values using a continuous number line thru two's complement representation;
 - (b) utilizing wider intermediate fractions during calculations to provide residual bits for rounding decisions;
 - (c) implementing rounding modes thru consistent mechanisms that work uniformly for both positive and negative values:
 - i. Round down by discarding residual bits;
 - ii. Round up by adding 1 to the lowest retained bit if any residual bits are non-zero;
 - iii. Round to nearest by adding 1 to the lowest retained bit if the high residual bit is 1;
 - (d) applying the same rounding logic regardless of value sign, thereby eliminating separate rounding paths based on sign testing.

16. A system for maintaining mathematical identities in floating-point operations, comprising:

(a) a processor; and

(b) a memory storing instructions which, when executed by the processor, implement:

i. a representation mechanism that:

- A. encodes different numerical states using specific normalization levels in two's complement fractions;
- B. uses a single reserved exponent value to identify values outside normal representation;
- C. distinguishes zero from vanished values and infinity from exploded values thru distinct bit patterns;

ii. arithmetic operations that:

- A. align fractions by checked exponent adjustment when performing addition or subtraction, transitioning to exploded states rather than infinity when overflow occurs and to vanished states rather than zero when underflow occurs, thereby ensuring $a - b = 0$ if and only if $a = b$, and $MAX + MAX \neq \infty$;
- B. multiply fractions while adding exponents, transitioning to exploded states rather than infinity when overflow occurs and to vanished states rather than zero when underflow occurs, thereby ensuring $a \times b = 0$ if and only if $a = 0$ or $b = 0$, and $MAX \times MAX \neq \infty$;
- C. divide fractions while subtracting exponents, transitioning to exploded states rather than infinity when overflow occurs and to vanished states rather than zero when underflow occurs, thereby ensuring non-zero $\div$ non-zero never produces zero or infinity, non-zero $\div 0$ produces infinity, and $0 \div$ non-zero produces zero;

iii. state transition mechanisms that:

- A. transition to N-1 normalized exploded states when exceeding maximum representable magnitude;
- B. transition to N-2 normalized vanished states when falling below minimum representable magnitude;
- C. preserve sign/orientation information in the most significant bits during transitions.

Figure 1: IEEE-754 Sign-Magnitude Representation

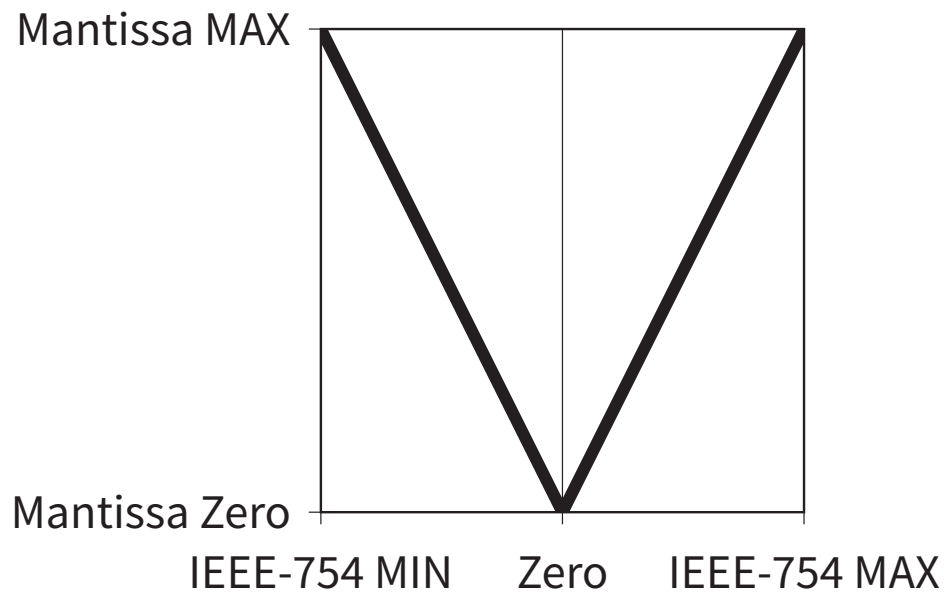


Figure 2: Two's Complement Representation

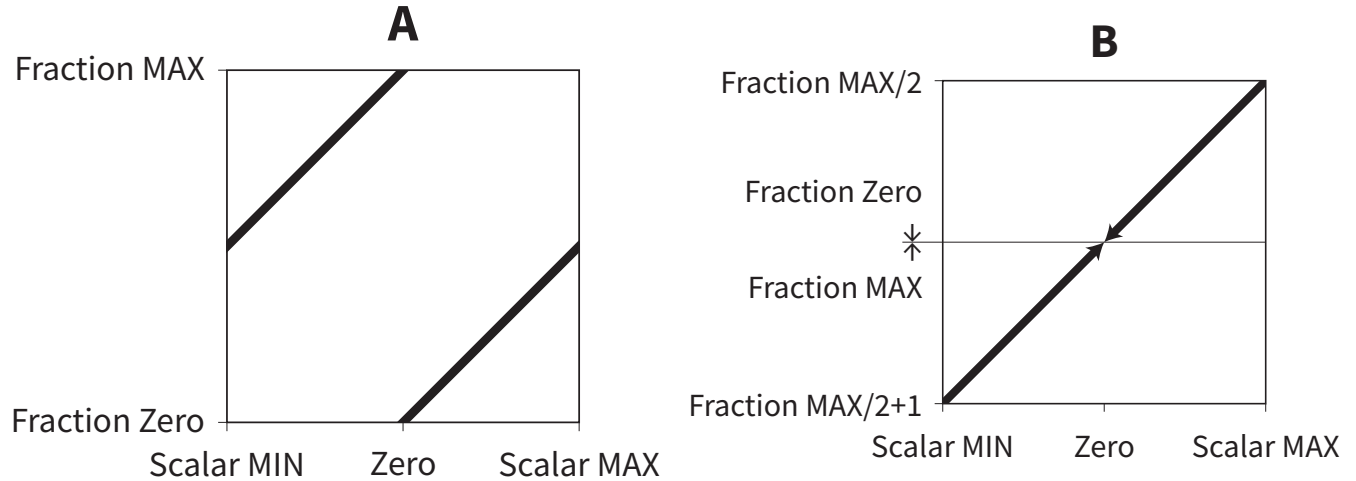


Figure 3: Scalar Type Definition in Rust

```
pub struct Scalar<F: Integer, E: Integer> {
    /// The normalized fraction component representing the significand.
    /// The fraction size determines the precision of the value.
    /// The N (prefix bit) pattern determines the number's state
    /// (normal, exploded, vanished, Zero or undefined/unimplemented).
    pub fraction: F,

    /// The exponent component determining the scale of the value/phase.
    /// The exponent size determines the range of the value/phase.
    /// When equal to AMBIGUOUS_EXPONENT, indicates an escaped phase
    /// (exploded, vanished, undefined/unimplemented or Zero).
    pub exponent: E,
}
```

Figure 4: Circle Type Definition in Rust

```
pub struct Circle<F: Integer, E: Integer> {
    /// The real phase representing the real part of the complex number.
    /// The normalization level of this component indicates current state
    /// (normal, exploded, vanished, undefined/unimplemented, Zero).
    pub real: F,

    /// The imaginary component representing the phase of i.
    /// The normalization level of this component indicates current state
    /// (normal, exploded, vanished, undefined/unimplemented, Zero).
    pub imaginary: F,

    /// The shared exponent component determining the scale of both phases.
    /// When equal to AMBIGUOUS_EXPONENT, indicates an escaped phase
    /// (exploded, vanished, undefined or Zero).
    pub exponent: E,
}
```

Figure 5: Where fraction: F and exponent: E are any sized integer

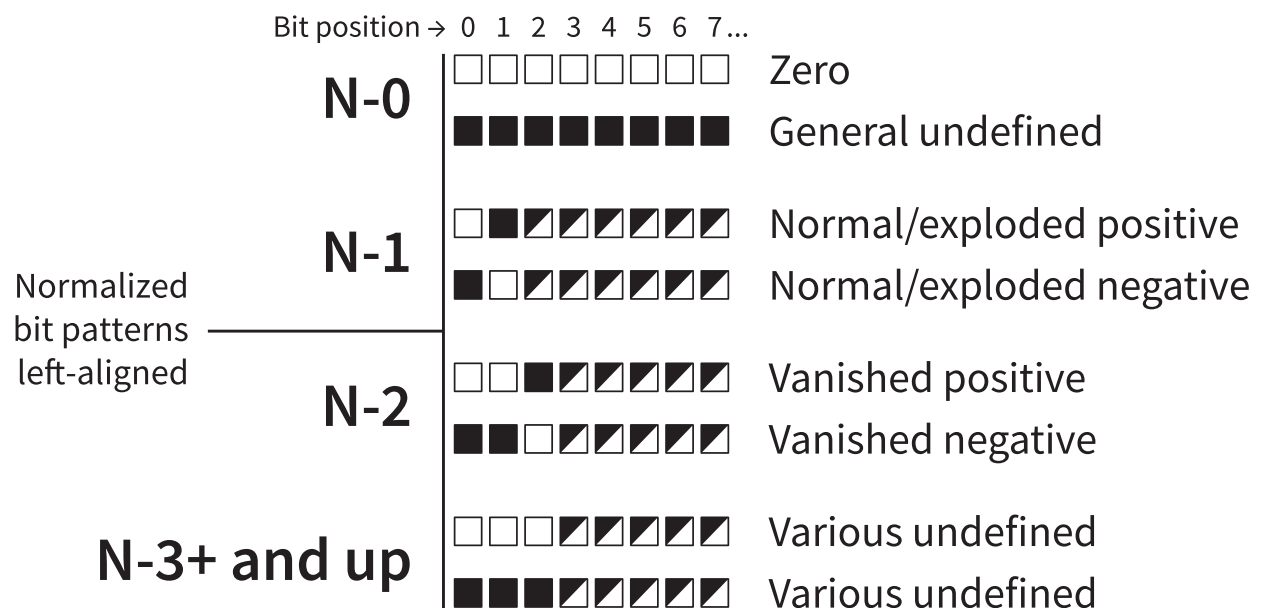
```
impl Integer for i8 {}
impl Integer for i16 {}
impl Integer for i32 {}
impl Integer for i64 {}
impl Integer for i128 {}
```

Figure 6: FE Type Configurations - Precision vs. Range

Range ↓	Precision (Fraction Bits)				
	F3 2.1 digits	F4 4.5 digits	F5 9.3 digits	F6 18.9 digits	F7 38.2 digits
E3 $\approx 10^{38.5}$	F3E3 ⟨i8, i8⟩	F4E3 ⟨i16, i8⟩	F5E3 ⟨i32, i8⟩	F6E3 ⟨i64, i8⟩	F7E3 ⟨i128, i8⟩
E4 $\approx 10^{9860}$	F3E4 ⟨i8, i16⟩	F4E4 ⟨i16, i16⟩	F5E4 ⟨i32, i16⟩	F6E4 ⟨i64, i16⟩	F7E4 ⟨i128, i16⟩
E5 $\approx 10^{(10^{8.81})}$	F3E5 ⟨i8, i32⟩	F4E5 ⟨i16, i32⟩	F5E5 ⟨i32, i32⟩	F6E5 ⟨i64, i32⟩	F7E5 ⟨i128, i32⟩
E6 $\approx 10^{(10^{18.4})}$	F3E6 ⟨i8, i64⟩	F4E6 ⟨i16, i64⟩	F5E6 ⟨i32, i64⟩	F6E6 ⟨i64, i64⟩	F7E6 ⟨i128, i64⟩
E7 $\approx 10^{(10^{37.7})}$	F3E7 ⟨i8, i128⟩	F4E7 ⟨i16, i128⟩	F5E7 ⟨i32, i128⟩	F6E7 ⟨i64, i128⟩	F7E7 ⟨i128, i128⟩

Figure 7: Normalization Levels and Bit Patterns

Two's Complement Fraction Representation



Legend: □ = 0 ■ = 1 ▨ = variable

Figure 8: Excerpt of Undefined State Definitions in Rust

```

// L3 Basic operators
/// ■■■■■■■■■■
pub const EXPLODED_PLUS_EXPLODED: Undefined = Undefined {
    prefix: 0b00011111u8 as i8,
    symbol: "⌘ ↑↑",
    description: "Exploded value addition with exploded value",
};
/// ■■■■■■■■■■
pub const EXPLODED_MINUS_EXPLODED: Undefined = Undefined {
    prefix: 0b11100000u8 as i8,
    symbol: "⌘ ↑-↑",
    description: "Exploded value subtraction with exploded value",
};
/// ■■■■■■■■■■
pub const VANISHED_PLUS_VANISHED: Undefined = Undefined {
    prefix: 0b00011110u8 as i8,
    symbol: "⌘ ↓+↓",
    description: "Vanished value addition with vanished value",
};
/// ■■■■■■■■■■
pub const VANISHED_MINUS_VANISHED: Undefined = Undefined {
    prefix: 0b11100001u8 as i8,
    symbol: "⌘ ↓-↓",
    description: "Vanished value subtraction with vanished value",
};

// L4 Powers and roots
/// ■■■■■■■■■■
pub const EXPLODED_POWER: Undefined = Undefined {
    prefix: 0b00001111u8 as i8,
    symbol: "⌘ ↑^",
    description: "Exploded value raised to power",
};
/// ■■■■■■■■■■
pub const VANISHED_POWER: Undefined = Undefined {
    prefix: 0b11110000u8 as i8,
    symbol: "⌘ ↓^",
    description: "Vanished value raised to power",
};

```

```

/// □□□□■□
pub const POWER_EXPLODED: Undefined = Undefined {
    prefix: 0b000011110u8 as i8,
    symbol: "⌘ ^↑",
    description: "Value raised to exploded power",
};

// L5 Logarithms
/// □□□□■□
pub const EXPLODED_LOG: Undefined = Undefined {
    prefix: 0b00001111u8 as i8,
    symbol: "⌘ ↑",
    description: "Logarithm of exploded value",
};

/// ■■■■□□
pub const VANISHED_LOG: Undefined = Undefined {
    prefix: 0b11111000u8 as i8,
    symbol: "⌘ ↓",
    description: "Logarithm of vanished value",
};

/// □□□□■□
pub const LOG_EXPLODED: Undefined = Undefined {
    prefix: 0b000011110u8 as i8,
    symbol: "⌘ Ⓜ↑",
    description: "Logarithm with exploded base",
};

// L6 Sine/Cosine
/// □□□□■□
pub const SINE: Undefined = Undefined {
    prefix: 0b00001111u8 as i8,
    symbol: "⌘ s",
    description: "Sine of value with imprecise period position",
};

/// ■■■■□□
pub const COSINE: Undefined = Undefined {
    prefix: 0b11111000u8 as i8,
    symbol: "⌘ c",
    description: "Cosine of value with imprecise period position",
};

```

Figure 9: SPIRIX Addition Operation Truth Table

$+$	0	$\downarrow$	#	$\uparrow$	∞	$\wp$
0	0	$\downarrow$	#	$\wp \uparrow +$	$\wp \uparrow +$	$\wp$
$\downarrow$	$\downarrow$	$\wp \downarrow + \downarrow$	#	$\wp \uparrow +$	$\wp \uparrow +$	$\wp$
#	#	#	0, $\downarrow$, #, $\uparrow$	$\wp \uparrow +$	$\wp \uparrow +$	$\wp$
$\uparrow$	$\wp \uparrow +$	$\wp \uparrow +$	$\wp \uparrow +$	$\wp \uparrow + \uparrow$	$\wp \uparrow + \uparrow$	$\wp$
∞	$\wp \uparrow +$	$\wp \uparrow +$	$\wp \uparrow +$	$\wp \uparrow + \uparrow$	$\wp \uparrow + \uparrow$	$\wp$
$\wp$	$\wp$	$\wp$	$\wp$	$\wp$	$\wp$	$\wp$

Figure 10: SPIRIX Subtraction Operation Truth Table

		Minuend					
Subtrahend	-	0	↓	#	↑	∞	⊘
	0	0	↓	#	⊘ ↑-	⊘ ↑-	⊘
	↓	↓	⊘ ↓-↓	#	⊘ ↑-	⊘ ↑-	⊘
	#	#, ↓, ↑	#, ↓, ↑	0, ↓, #, ↑	⊘ ↑-	⊘ ↑-	⊘
	↑	⊘ -↑	⊘ -↑	⊘ -↑	⊘ ↑-↑	⊘ ↑-↑	⊘
	∞	⊘ -↑	⊘ -↑	⊘ -↑	⊘ ↑-↑	⊘ ↑-↑	⊘
	⊘	⊘	⊘	⊘	⊘	⊘	⊘

Figure 11: SPIRIX Multiplication Operation Truth Table

$\times$	0	$\downarrow$	#	$\uparrow$	∞	$\emptyset$
0	0	0	0	0	$\emptyset \uparrow \times \downarrow$	$\emptyset$
$\downarrow$	0	$\downarrow$	$\downarrow$	$\emptyset \uparrow \times \downarrow$	∞	$\emptyset$
#	0	$\downarrow$	#, $\downarrow, \uparrow$	$\uparrow$	∞	$\emptyset$
$\uparrow$	0	$\emptyset \downarrow \times \uparrow$	$\uparrow$	$\uparrow$	∞	$\emptyset$
∞	$\emptyset \downarrow \times \uparrow$	∞	∞	∞	∞	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Figure 12: SPIRIX Division Operation Truth Table

		Numerator					
Denominator	/	0	↓	#	↑	∞	⊘
	0	⊘ ↓/↓	∞	∞	∞	∞	⊘
	↓	0	⊘ ↓/↓	↑	↑	∞	⊘
	#	0	↓	#, ↓, ↑	↑	∞	⊘
	↑	0	↓	↓	⊘ ↑/↑	∞	⊘
	∞	0	0	0	0	⊘ ↑/↑	⊘
	⊘	⊘	⊘	⊘	⊘	⊘	⊘

Figure 13: SPIRIX Number Line and Comparison Order

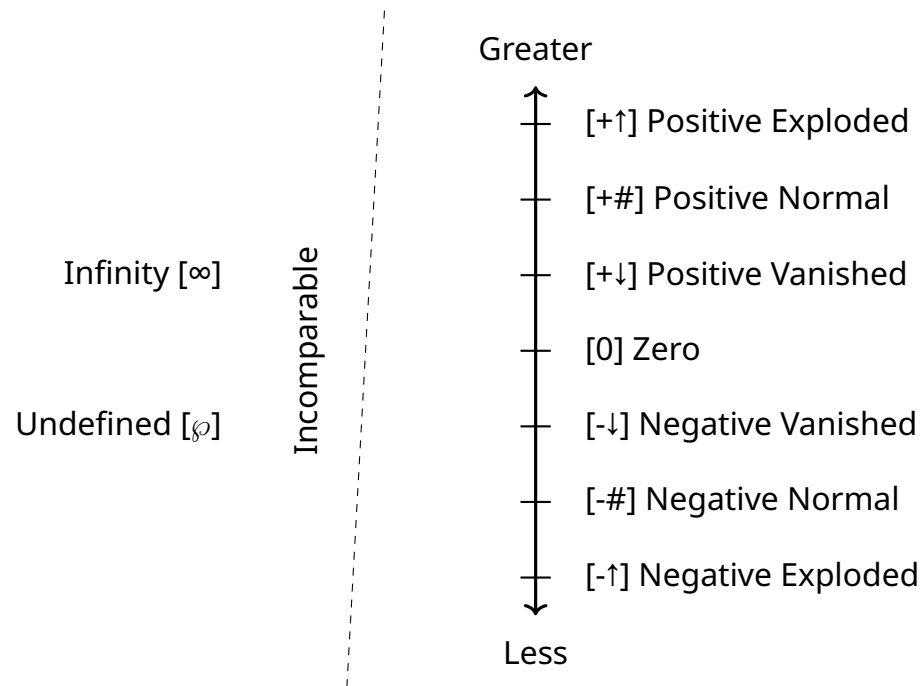
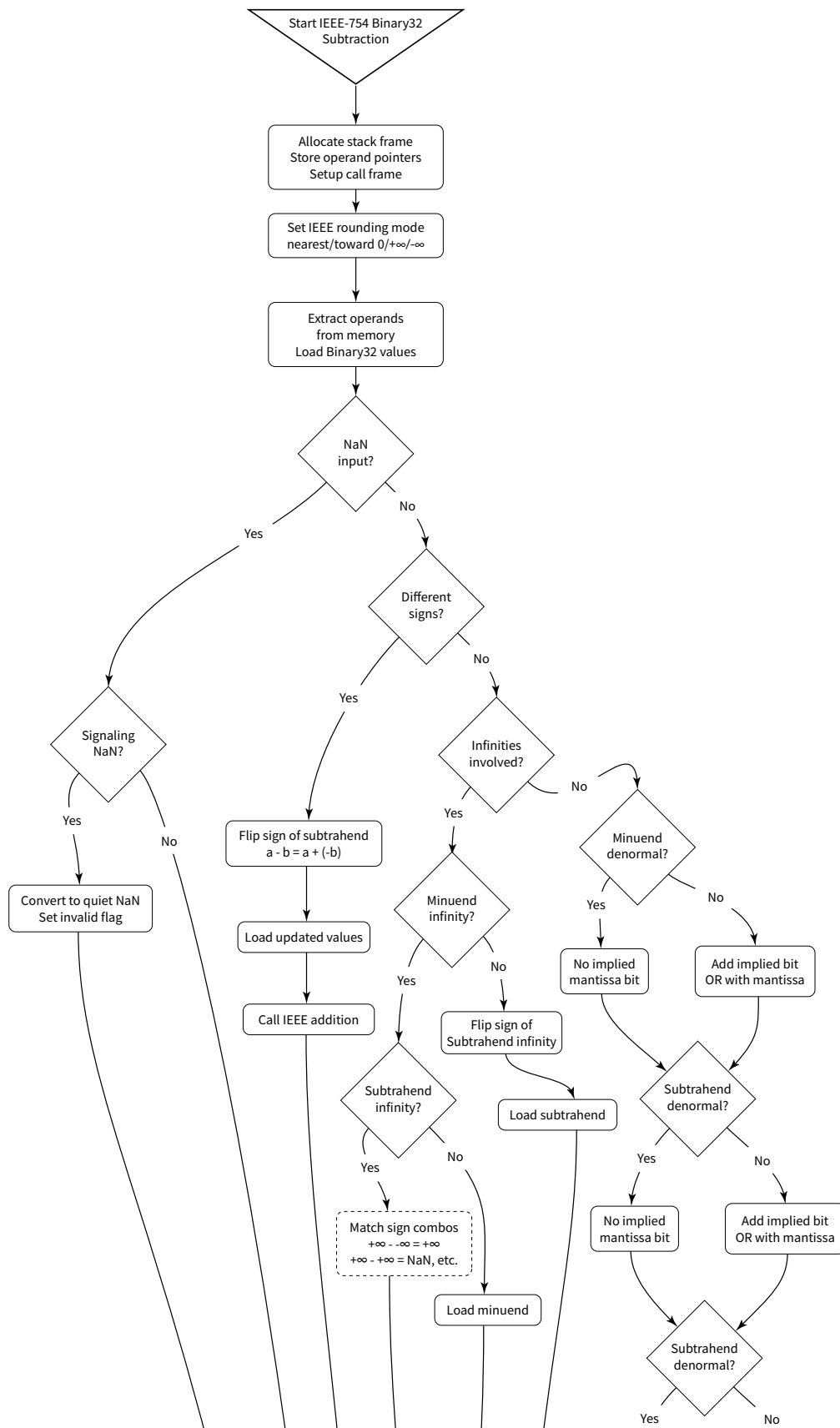
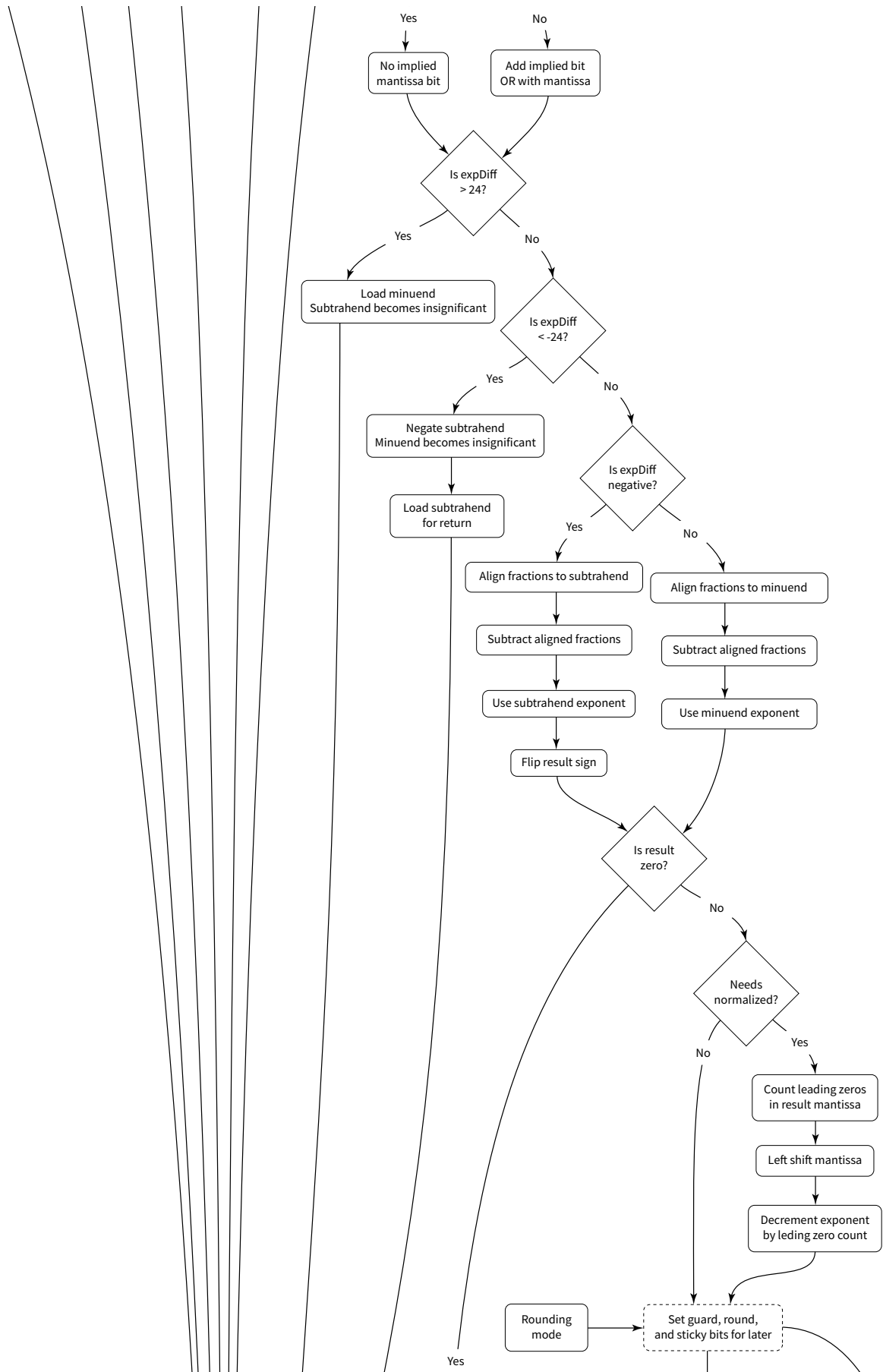


Figure 14: IEEE-754 Binary32 Subtraction Flowchart





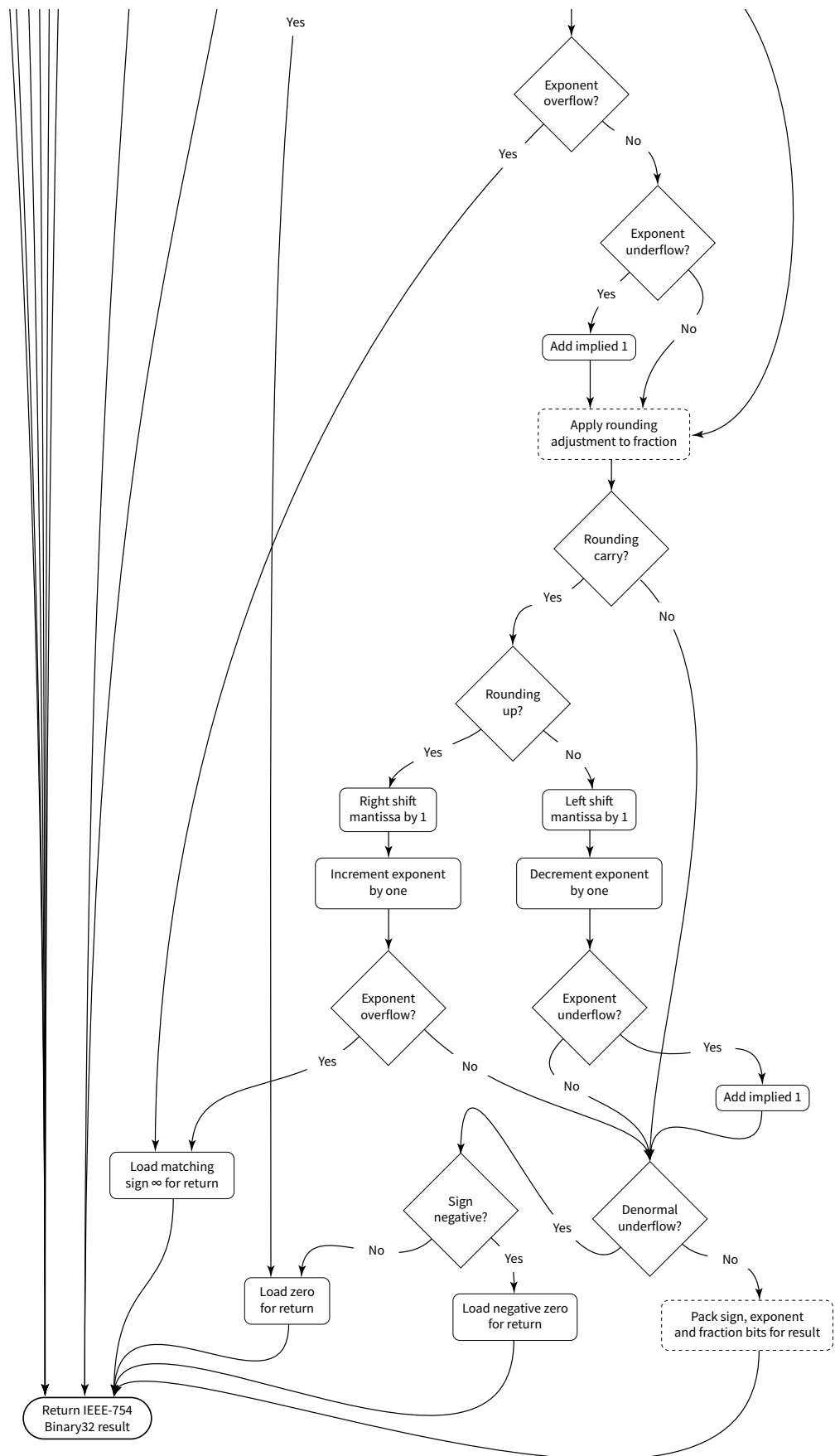


Figure 15: Scalar Subtraction Flowchart

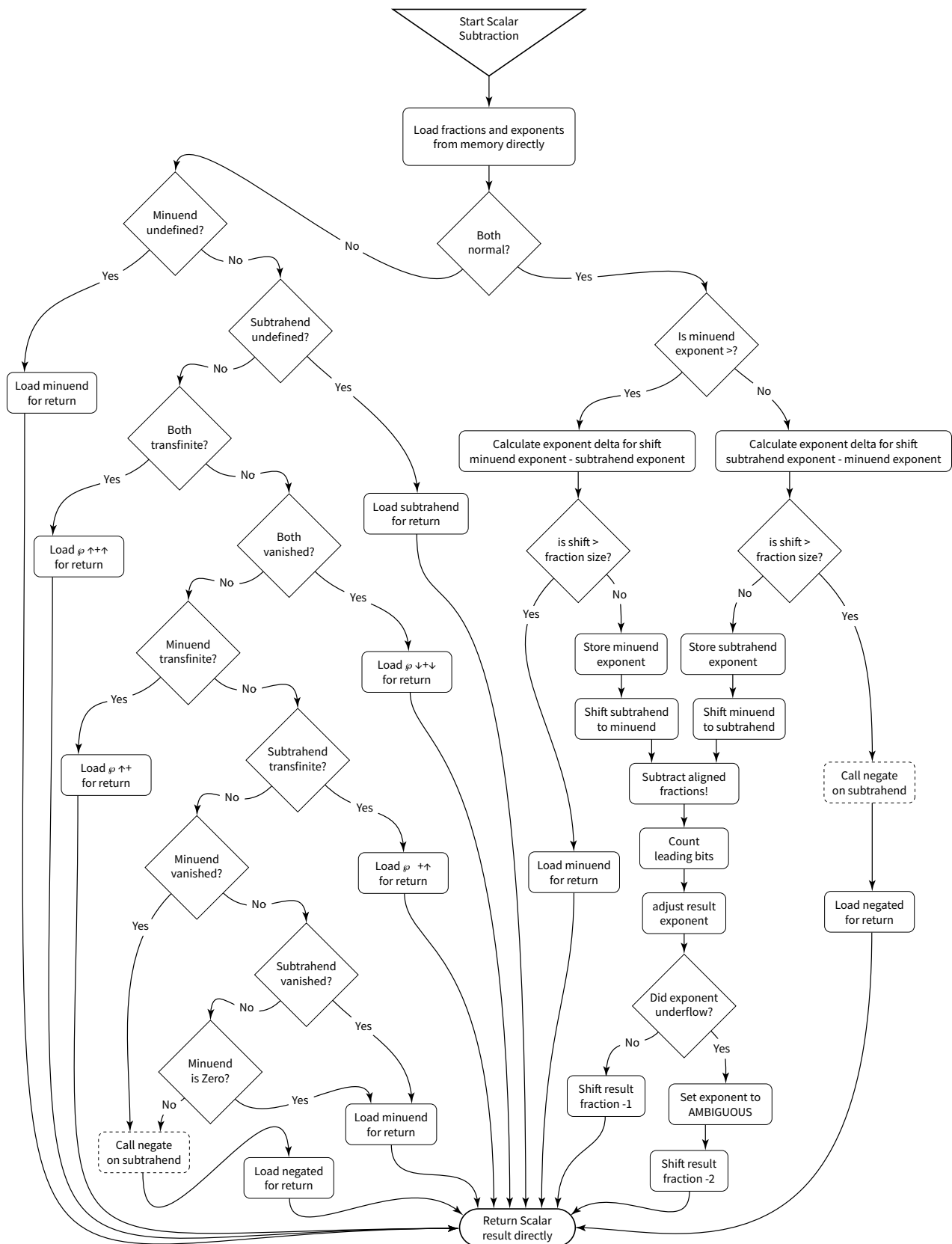


Figure 16: Annotated IEE-754 Binary32 Normal Subtraction ASM Trace

```

; Main Subtraction Fuction Entry Point
[001]sub rsp, 0x58                ; Allocate 88 bytes on the stack
[002]mov qword ptr [rsp 0x8], rdi ; Store first parameter (pointer to first value)
[003]mov al, dl                   ; Move the 3rd parameter (lower byte) to al
[004]mov dword ptr [rsp 0x18], esi ; Store second parameter (second value)
[005]mov ecx, dword ptr [rsp 0x18] ; Load second Binary32 value into ecx
[006]mov dword ptr [rsp 0x14], ecx ; Store second Binary32 value at rsp+20
[007]mov byte ptr [rsp 0x1f], al   ; Store lower byte of 3rd parameter at rsp+31
[008]mov qword ptr [rsp 0x30], rdi ; Move first parameter
[009]lea rax, ptr [rip 0x3d6e9]    ; Load effective address
[010]lea rdi, ptr [rsp 0x1f]       ; Load address of next function into rdi
[011]call rax                     ; Call function at address in rax

; Set Rounding Mode
[012]push rax                     ; Save rax register
[013]mov qword ptr [rsp], rdi     ; Store rdi at top of stack
[014]call 0x558bdf671e20          ; Call function to set rounding mode

; Convert Rounding Mode to Float Format
[015]mov qword ptr [rsp-0x8], rdi ; Store rdi at rsp-8
[016]movzx eax, byte ptr [rdi]    ; Zero-extend byte at [rdi] into eax
[017]mov qword ptr [rsp-0x18], rax ; Store extended value at rsp-24
[018]mov rax, qword ptr [rsp-0x18] ; Load value from rsp-24 into rax
[019]lea rcx, ptr [rip 0x452af]    ; Load effective address (jump table?)
[020]movsxd rax, dword ptr [rcx rax*4] ; Load and sign-extend value from jump table
[021]add rax, rcx                  ; Add base address to jump offset
[022]jmp rax                       ; Jump to the calculated address
[023]mov byte ptr [rsp-0x9], 0x0   ; Set byte at rsp-9 to 0
[024]jmp 0x558bdf671e65           ; Jump to function epilogue
[025]mov al, byte ptr [rsp-0x9]    ; Load byte from rsp-9 into al
[026]ret                           ; Return from function

; Set Rounding Mode
[027]movzx edi, al                 ; Zero-extend al into edi
[028]call qword ptr [rip 0x5bf45] ; Call function pointer at rip+0x5bf45

; Rounding Mode Helper
[029]push rbp                     ; Save base pointer
[030]mov rbp, rsp                 ; Set new base pointer

```

```

[031]push rbx                ; Save rbx register
[032]sub rsp, 0x8            ; Allocate 8 bytes on stack
[033]mov ebx, edi            ; Store first parameter in ebx
[034]mov rax, qword ptr fs:[0x0] ; Load value from fs segment
[035]lea rax, ptr [rax-0x4f]  ; Calculate address at fs:0 - 79
[036]mov byte ptr [rax], bl   ; Store lowest byte of ebx at calculated address
[037]mov rbx, qword ptr [rbp-0x8] ; Restore rbx
[038]leave                  ; Restore stack and base pointer
[039]ret                    ; Return from function

; Set Rounding Mode (continued)
[040]pop rax ; Restore rax
[041]ret     ; Return from function

; Main Subtraction Function (continued)
[042]jmp 0x558bdf63473a      ; Jump to next part of subtraction function
[043]mov rax, qword ptr [rsp+0x8] ; Load pointer to first Binary32 from rsp+8
[044]mov eax, dword ptr [rax]   ; Load value of first Binary32 into eax
[045]mov dword ptr [rsp+0x28], eax ; Store first Binary32 at rsp+40
[046]lea rdi, ptr [rsp+0x14]    ; Load address of second value into rdi
[047]call 0x558bdf6346e0        ; Call function to borrow memory value

; Borrow Memory Value
[048]mov rax, rdi              ; Copy input parameter to rax
[049]mov qword ptr [rsp-0x8], rax ; Store rax at rsp-8
[050]ret                      ; Return from function

; Main Subtraction Function (continued)
[051]mov qword ptr [rsp], rax   ; Store return value at top of stack
[052]jmp 0x558bdf634755        ; Jump to next part of subtraction
[053]mov rax, qword ptr [rsp]   ; Load pointer from stack into rax
[054]mov eax, dword ptr [rax]   ; Load value at pointer into eax
[055]mov dword ptr [rsp+0x2c], eax ; Store second Binary32 at rsp+44
[056]mov eax, dword ptr [rsp+0x28] ; Load first Binary32 into eax
[057]mov dword ptr [rsp+0x4c], eax ; Store first Binary32 at rsp+76
[058]mov edi, dword ptr [rsp+0x4c] ; Load first value into edi
[059]mov eax, dword ptr [rsp+0x2c] ; Load second Binary32 into eax
[060]mov dword ptr [rsp+0x50], eax ; Store second Binary32 at rsp+80
[061]mov esi, dword ptr [rsp+0x50] ; Load second value into esi
[062]call qword ptr [rip+0x9906b] ; Call function for low-level

```

; Low-level Float Subtraction

```
[063]push rbp          ; Save base pointer
[064]mov rbp, rsp      ; Set new base pointer
[065]mov eax, edi      ; Copy first Binary32 to eax
[066]mov edx, esi      ; Copy second Binary32 to edx
[067]xor esi, edi      ; XOR the two Binary32 values
[068]js 0x558bdf671e83 ; Jump if sign bit is set (opposing signs)
[069]mov rsi, rdx      ; Copy second Binary32 to rsi
[070]mov rdi, rax      ; Copy first Binary32 to rdi
[071]call 0x558bdf6721a9 ; Call function to add float magnitudes
```

; Add Float Magnitudes

```
[072]push rbp          ; Save base pointer
[073]mov rbp, rsp      ; Set new base pointer
[074]mov rcx, rdi      ; Copy first Binary32 to rcx
[075]shr rcx, 0x17     ; Shift right by 23 bits (extract exponent)
[076]movzx ecx, cl      ; Zero-extend lowest byte to get 8-bit exponent
[077]mov r9, rcx       ; Copy exponent to r9
[078]mov r10, rdi      ; Copy first Binary32 to r10
[079]and r10d, 0x7fffff ; Mask to get mantissa (23 bits)
[080]mov rax, rsi      ; Copy second Binary32 to rax
[081]shr rax, 0x17     ; Shift right by 23 bits (extract exponent)
[082]movzx eax, al      ; Zero-extend lowest byte to get 8-bit exponent
[083]mov rdx, rsi      ; Copy second Binary32 to rdx
[084]and edx, 0x7fffff ; Mask to get mantissa (23 bits)
[085]mov r11, rcx       ; Copy first exponent to r11
[086]sub r11, rax       ; Subtract exponents (for comparing magnitude)
[087]jnz 0x558bdf67224f ; Jump if exponents are different
[088]mov r8d, edi      ; Copy first Binary32 to r8d
[089]shr r8d, 0x1f     ; Shift right by 31 bits (extract sign bit)
[090]shl r10, 0x6       ; Shift first mantissa left by 6 bits (normalize)
[091]shl rdx, 0x6       ; Shift second mantissa left by 6 bits (normalize)
[092]test r11, r11      ; Test if exponent difference is Zero
[093]js 0x558bdf6722c4 ; Jump if exponent difference is negative
[094]cmp rax, 0xff      ; Check for NaN/Infinity
[095]jz 0x558bdf67230b ; Jump if second number is NaN or Infinity
[096]test rcx, rcx      ; Test if first exponent is Zero
[097]mov esi, 0x200000000 ; Set esi to 0x200000000 (implied 1 for normal numbers)
[098]cmovz rsi, r10     ; True for normals, false for subnormal
```

```

[099]mov rdi, rax          ; Copy second exponent to rdi
[100]sub rdi, rcx          ; Calculate exponent difference
[101]add rsi, r10          ; Add implied 1 to first mantissa
[102]cmp rdi, 0x1e         ; Compare exponent difference with 30
[103]jnb 0x558bdf672327 ; Jump if difference is not within range
[104]mov ecx, edi          ; Copy exponent difference to ecx
[105]neg ecx              ; Negate exponent difference
[106]mov r9d, esi         ; Copy first mantissa with implied 1 to r9d
[107]shl r9d, cl          ; Shift left by exponent difference
[108]test r9d, r9d        ; Test if shifted value is Zero
[109]setnz r10b           ; Set r10b to 1 if not Zero, 0 if Zero
[110]movzx r10d, r10b     ; Zero-extend r10b to r10d
[111]mov ecx, edi         ; Copy exponent difference to ecx
[112]shr esi, cl          ; Shift first mantissa right by exponent difference
[113]or r10d, esi         ; Combine shifted bits with sticky bit
[114]mov r10d, r10d       ; Clear upper 32 bits of r10
[115]mov r9, rax          ; Copy second exponent to r9
[116]jmp 0x558bdf6722a3 ; Jump to next part of float addition
[117]lea rdx, ptr [r10 rdx*1 0x200000000] ; Calculate sum of mantissas
[118]cmp rdx, 0x3fffffff ; Check if result needs normalization
[119]jnb 0x558bdf67220e ; Jump if normalization needed
[120]movzx edi, r8b       ; Zero-extend sign bit to edi
[121]mov rsi, r9          ; Copy exponent to rsi
[122]call 0x558bdf671f65 ; Call function to round and pack to Binary32 format

; Round and Pack to Binary32 Format
[123]push rbp             ; Save base pointer
[124]mov rbp, rsp         ; Set new base pointer
[125]push r15             ; Save r15 register
[126]push r14             ; Save r14 register
[127]push r13             ; Save r13 register
[128]push r12             ; Save r12 register
[129]push rbx             ; Save rbx register
[130]sub rsp, 0x18        ; Allocate 24 bytes on stack
[131]mov r15d, edi        ; Store sign bit in r15d
[132]mov dword ptr [rbp-0x34], edi ; Store sign bit at rbp-52
[133]mov rbx, rsi         ; Store exponent in rbx
[134]mov r13, rdx         ; Store mantissa in r13
[135]mov rax, qword ptr fs:[0x0] ; Load value from fs segment
[136]lea rax, ptr [rax-0x4f] ; Calculate address at fs:0 - 79

```

```

[137]movzx r14d, byte ptr [rax]      ; Load rounding mode into r14d
[138]test r14b, r14b                ; Test if rounding mode is 0 (to nearest)
[139]setz byte ptr [rbp-0x40]        ; Set flag at rbp-64 if rounding mode is 0
[140]mov r12d, 0x40                 ; Set r12d to 64 (used for rounding)
[141]test r14b, 0xfb                ; Test rounding mode against 0xfb
[142]jz 0x558bdf671fc9              ; Jump if specific rounding mode
[143]mov r15d, r13d                 ; Copy mantissa to r15d
[144]and r15d, 0x7f                 ; Mask with 0x7f (7 least significant bits)
[145]mov dword ptr [rbp-0x38], ebx   ; Store exponent at rbp-56
[146]cmp ebx, 0xfc                  ; Compare exponent with 0xfc
[147]jbe 0x558bdf672000             ; Jump if exponent <= 0xfc
[148]movzx r12d, r12b               ; Zero-extend r12b to r12d
[149]add r12, r13                   ; Add mantissa to r12
[150]shr r12, 0x7                   ; Shift right by 7 bits (rounding)
[151]test r15b, r15b                ; Test if lower 7 bits are Zero
[152]jnz 0x558bdf6720ec             ; Jump if lower 7 bits are not Zero
[153]mov rax, qword ptr fs:[0x0]    ; Load value from fs segment
[154]lea rax, ptr [rax-0x50]         ; Calculate address at fs:0 - 80
[155]or byte ptr [rax], 0x1         ; Set inexact flag
[156]cmp r14b, 0x6                  ; Compare rounding mode with 6
[157]jnz 0x558bdf672014             ; Jump if rounding mode is not 6
[158]cmp r15b, 0x40                 ; Compare lower 7 bits with 0x40
[159]setz al                         ; Set al to 1 if equal, 0 if not
[160]movzx eax, al                  ; Zero-extend al to eax
[161]and rax, qword ptr [rbp-0x40]  ; AND with rounding flag
[162]not rax                         ; Complement result
[163]and r12, rax                   ; Mask r12 with result
[164]mov eax, 0x0                   ; Set eax to 0
[165]cmovz rbx, rax                 ; If Zero, set exponent to 0
[166]mov eax, dword ptr [rbp-0x34]  ; Load sign bit into eax
[167]shl eax, 0x1f                  ; Shift sign bit to position 31
[168]shl ebx, 0x17                  ; Shift exponent to position 23-30
[169]add eax, ebx                   ; Combine sign and exponent
[170]mov eax, eax                   ; Clear upper 32 bits
[171]add rax, r12                   ; Add mantissa to result
[172]add rsp, 0x18                  ; Deallocate stack space
[173]pop rbx                        ; Restore rbx
[174]pop r12                        ; Restore r12
[175]pop r13                        ; Restore r13
[176]pop r14                        ; Restore r14

```

```
[177]pop r15                ; Restore r15
[178]pop rbp                ; Restore base pointer
[179]ret                    ; Return from function

; Add Float Magnitudes (epilogue)
[180]pop rbp ; Restore base pointer
[181]ret      ; Return from function

; Low-level Float Subtraction (epilogue)
[182]jmp 0x558bdf671e81 ; Jump to another part of function
[183]pop rbp          ; Restore base pointer
[184]ret              ; Return from function

; Main Subtraction Function (epilogue)
[185]mov dword ptr [rsp 0x54], eax ; Store result at rsp+84
[186]mov eax, dword ptr [rsp 0x54] ; Load result back into eax
[187]mov dword ptr [rsp 0x24], eax ; Store result at rsp+36
[188]mov eax, dword ptr [rsp 0x24] ; Load result back into eax
[189]mov dword ptr [rsp 0x20], eax ; Store result at rsp+32
[190]mov eax, dword ptr [rsp 0x20] ; Load result back into eax
[191]add rsp, 0x58                ; Deallocate 88 bytes from stack
[192]ret                          ; Return from function
```

Figure 17: Assembly Implementation of Scalar Subtraction

```

; ScalarF5E4-ScalarF5E4 by Nick Spiker fractaldecoder@proton.me
; Performs normal subtraction with 32-bit fractions and 16-bit exponents
; Input: Result pointer in rdi, Minuend in rsi, Subtrahend in rdx

; Register allocation:
; rax - a.fraction/result fraction (64-bit extended)
; rbx - b.fraction (64-bit extended)
; ecx - exponent difference/normalization shift
; r8d - a.exponent/result exponent
; r9d - b.exponent
; edx - leading bits counter

; Load values with sign extension
[01]movsx rax, dword ptr [rsi] ; Load a.fraction
[02]movsx rbx, dword ptr [rdx] ; Load b.fraction
[03]movsx r8d, word ptr [rsi+4] ; Load a.exponent
[04]movsx r9d, word ptr [rdx+4] ; Load b.exponent

; Check for ambiguous exponents
[05]cmp r8d, 0x8000 ; Compare with AMBIGUOUS_EXPONENT
[06]je ambiguous_handler ; Jump if a is ambiguous
[07]cmp r9d, 0x8000 ; Compare with AMBIGUOUS_EXPONENT
[08]je ambiguous_handler ; Jump if b is ambiguous

; Calculate exponent difference
[09]mov ecx, r8d ; Use ecx to store exponent diff
[10]sub ecx, r9d ; ecx = a.exp - b.exp
[11]js b_larger ; Jump if b's exponent is larger

; Minuend has larger exponent
[12]cmp ecx, 32 ; Shift amount >= 32 bits?
[13]jge return ; If yes, b's contribution is negligible

; Perform shift and subtraction
[14]shl rax, cl ; Align a to b
[15]sub rax, rbx ; (a << diff) - b
[16]test rax, rax ; Check if result is zero
[17]jz set_ambiguous ; If zero, set exponent to AMBIGUOUS
[18]lea r8d, [r9d+32] ; Intermediary exponent=b.exp+offset

```

normalize:

```
[19]lzcnt edx, eax      ; Count leading zeros
[20]not rax             ; Invert for leading ones count
[21]lzcnt ecx, eax      ; Count leading ones (as zeros in inverted)
[22]not rax             ; Restore original value
[23]cmp edx, ecx        ; Compare leading zeros with leading ones
[24]cmovl edx, ecx      ; Take max (Leading Same Bits)
[25]dec edx             ; Offset for N-1 normalization
[26]shl rax, edx        ; Normalize to N-1 high bits
[27]shr rax, 32         ; Re-align to single-wide
[28]sub r8d, edx        ; Adjust result exponent by shift amount
[29]cmp r8d, 0x8001     ; Check result for underflow
[30]jl underflow_handler ; Jump if underflow detected
```

return:

```
[31]mov dword ptr [rdi], eax ; Store result fraction
[32]mov word ptr [rdi+4], r8w ; Store result exponent
[33]ret
```

underflow_handler:

```
[34]shr rax, 1 ; Shift fraction to N-2 normalization
```

set_ambiguous:

```
[35]mov r8d, 0x8000 ; Set exponent to AMBIGUOUS_EXPONENT
[36]jmp return
```

b_larger:

```
[37]neg ecx          ; Convert negative diff to positive
[39]jge b_dominates ; If yes, a's contribution is negligible
```

; Perform shift and subtraction with b larger

```
[40]shl rbx, cl      ; Align b to a
[41]sub rax, rbx      ; a - (b << diff)
[42]test rax, rax     ; Check if result is zero
[43]jz set_ambiguous ; If zero, set exponent to AMBIGUOUS
[44]add r8d, 32       ; Set up for normalization alignment
[45]jmp normalize
```

b_dominates:

```
[46]mov rax, rbx      ; Result fraction is just -b
[47]neg rax           ; Negate in double-wide to avoid MIN issues
[48]lea r8d, [r9d+32] ; Use b's exponent with normalization offset
[49]jmp normalize

; Implementation of ambiguous_handler would follow, similar to figure 18, [073]
```

Figure 18: Complete Complex Division Assembly Implementation

```

; CircleF5E4/CircleF5E4 by Nick Spiker fractaldecoder@proton.me
; Performs complex division with 32-bit fractions and 16-bit exponents
; Input: Result pointer in rdi, Dividend in rsi, Divisor in rdx
;  $(a + bi) / (c + di) = [(a*c + b*d) + (b*c - a*d)i] * 1 / (c*c + d*d)$ 

; Register allocation:
[001] movsx r8d, dword ptr [rsi] ; Load a (numerator real fraction)
[002] movsx r9d, dword ptr [rsi+4] ; Load b (numerator imaginary fraction)
[003] movsx r10d, dword ptr [rdx] ; Load c (denominator real fraction)
[004] movsx r11d, dword ptr [rdx+4] ; Load d (denominator imaginary fraction)
[005] movsx di, word ptr [rsi+8] ; Load numerator exponent
[006] movsx si, word ptr [rdx+8] ; Load denominator exponent

; Check both exponents for ambiguity
[007] cmp di, 0x8000 ; Compare with AMBIGUOUS_EXPONENT
[008] je ambiguous_handler ; Jump if numerator ambiguous
[009] cmp si, 0x8000 ; Compare with AMBIGUOUS_EXPONENT
[010] je ambiguous_handler ; Jump if denominator ambiguous

; Calculate result exponent (numerator exponent - denominator exponent)
[011] sub edi, esi ; di - si (double width, lands in edi)
[012] xor esi, esi ; Zero esi to use as normal(0)/exploded(1)/vanished(2) flag

main_path:
; Calculate denominator magnitude squared (c*c + d*d)
[013] mov eax, r10d ; Copy c for squaring
[014] mul eax ; Square!
[015] mov ecx, edx ; Pull left half
[016] mov eax, r11d ; Copy d for squaring
[017] mul eax ; Square!
[018] add ecx, edx ; Add both high halves, mag squared is now in ecx
[019] mov rax, 0x4000000000000000 ; Left-aligned 1
[020] xor rdx, rdx ; Clear numerator left half (128-bit divide)
[021] div ecx ; Denominator is now in eax

; Calculate real numerator = a*c + b*d
[022] imul rcx, r8, r10 ; a*c 64-bit multiply
[023] imul rdx, r9, r11 ; b*d
[024] shr rcx, 1 ; Prepare for adding to avoid overflow

```

```

[025] shr rdx, 1
[026] add rcx, rdx      ; real numerator is now in rcx

; Calculate imaginary numerator = b*c - a*d
[027] imul r9, r10
[028] imul r8, r11
[029] shr r9, 1
[030] shr r8, 1
[031] sub r9, r8 ; imaginary numerator is now in r9

; Calculate r and i quotient
[032] imul r8, rcx, eax ; multiply real by reciprocal
[033] imul r9, eax      ; same for i

; Find max(leading_zeros, leading_ones) for real result fraction
[034] lzcnt rax, r8 ; Count leading Zeros
[035] not r8        ; Invert bits
[036] lzcnt rdx, r8 ; Count leading ones (as Zeros in inverted)
[037] not r8        ; Invert back to original value
[038] cmp rax, rdx  ; Compare leading Zeros with leading ones
[039] cmovl rax, rdx ; If rax < rdx, move rdx to rax

; Find max(leading_zeros, leading_ones) for imaginary result fraction
[040] lzcnt rcx, r9
[041] not r9
[042] lzcnt rdx, r9
[043] not r9
[044] cmp rcx, rdx
[045] cmovl rcx, rdx

; Find min(max_real, max_imag) and store in rax
[046] cmp rax, rcx ; Compare max_real with max_imag
[047] cmovg rax, rcx ; If rax > rcx, move rcx to rax
[048] dec rax      ; N-1 normalization

[049] shl r8, rax ; Normalize real
[050] shr r8, 32  ; Align to 32-bit fraction
[051] shl r9, rax ; Normalize i
[052] shr r9, 32

```

```

[053] test esi, esi ; Check flag for escaped
[054] jnz .escaped ; Wee!

[055] sub di, rax ; Adjust exponent from normalization shift

; Check adjusted exponent for over/underflow
[056] cmp di, 0x7FFF ; Compare with MAX_EXPONENT
[057] jg .exploded ; Jump if di > MAX_EXPONENT
[058] cmp di, 0x8001 ; Compare with MIN_EXPONENT
[059] jl .vanished ; Jump if di < MIN_EXPONENT

return:
[060] mov dword ptr [rdi], r8d ; Store real fraction
[061] mov dword ptr [rdi+4], r9d ; Store imaginary fraction
[062] mov word ptr [rdi+8], di ; Store exponent
[063] ret

vanished:
[064] shr r8, 1 ; Shift real fraction right by 1
[065] shr r9, 1 ; Shift imaginary fraction right by 1

exploded:
[066] mov di, 0x8000 ; Set to ambiguous exponent

[067] jmp return

escaped:
[068] shr esi, 1 ; Convert flag to shift
[069] shr r8, esi
[070] shr r9, esi
[071] mov di, 0x8000
[072] jmp return

ambiguous_handler:
; Check if numerator's exponent is ambiguous
[073] cmp di, 0x8000 ; Is numerator exponent ambiguous?
[074] jne .check_denom_ambiguous ; If not, check if denominator is ambiguous

; Check if numerator is undefined (ambiguous exponent + undefined bit pattern)
[075] cmp r8d, r9d ; Compare real and imaginary fractions

```

```

[076] jne .check_denom_ambiguous ; If they don't match, not undefined!

[077] test r8d, r8d                ; Is real part Zero?
[078] jz .check_denom_ambiguous ; If Zero, not undefined

; Check if top 3 bits are equal (undefined or Zero pattern)
[079] mov eax, r8d                ; Get real part
[080] shr eax, 29                 ; Isolate top 3 bits
[081] mov ecx, eax                ; Copy top bits
[082] ror ecx, 1                  ; Rotate right by 1
[083] cmp eax, ecx                ; Are top 3 bits uniform?
[084] je .return_numerator ; If uniform (undefined), return numerator

.check_denom_ambiguous:
; Check if denominator is undefined
[085] cmp si, 0x8000 ; Is denominator exponent ambiguous?
[086] jne .check_ambiguous_cases

[087] cmp r10d, r11d             ; Compare real and imaginary parts
[088] jne .check_ambiguous_cases ; If they don't match, not undefined

[089] test r10d, r10d ; Is real part Zero?
[090] jz .check_zero ; If Zero, might be division by Zero

; Check top 3 bits for undefined pattern
[091] mov eax, r10d                ; Get real part
[092] shr eax, 29                 ; Isolate top 3 bits
[093] mov ecx, eax                ; Copy top bits
[094] ror ecx, 1                  ; Rotate right by 1
[095] cmp eax, ecx                ; Are top 3 bits uniform? (undefined check)
[096] je .return_denominator ; If undefined, return denominator

[097] jmp .check_ambiguous_cases

.check_zero:
; Check if denominator is Zero - both parts are Zero
[098] test r11d, r11d            ; Test if imaginary part is also Zero
[099] jnz .check_ambiguous_cases ; If not Zero, check other cases

; Return division by Zero undefined value

```

```

[100] mov eax, 0x00010111      ; Load DIVIDE_BY_ZERO prefix
[101] mov dword ptr [rdi], eax  ; Store real component with prefix
[102] mov dword ptr [rdi+4], eax ; Store imaginary component with prefix
[103] mov word ptr [rdi+8], 0x8000 ; Set exponent to AMBIGUOUS_EXPONENT
[104] ret

```

.check_ambiguous_cases:

; Check for both exploded

```

[105] mov eax, r8d
[106] shr eax, 30      ; Get top 2 bits
[107] cmp eax, 0x01 ; Check for □■ pattern (exploded positive)
[108] je .check_numerator_exploded
[109] cmp eax, 0x02 ; Check for ■□ pattern (exploded negative)
[110] jne .check_numerator_vanished

```

.check_numerator_exploded:

```

[111] mov eax, r10d
[112] shr eax, 30      ; Get top 2 bits
[113] cmp eax, 0x01 ; Check for □■ pattern (exploded positive)
[114] je .exploded_divide_exploded
[115] cmp eax, 0x02 ; Check for ■□ pattern (exploded negative)
[116] je .exploded_divide_exploded

```

; Check if denominator is vanished

```

[117] mov eax, r10d
[118] shr eax, 29      ; Get top 3 bits
[119] cmp eax, 0x01 ; Check for □□■ pattern (vanished positive)
[120] je .exploded_divide_vanished
[121] cmp eax, 0x06 ; Check for ■■■□ pattern (vanished negative)
[122] je .exploded_divide_vanished

```

; Numerator exploded, denominator normal

; Set flag for exploded (1) and go to main_path

```

[123] mov esi, 1
[124] jmp main_path

```

.exploded_divide_vanished:

; Numerator exploded, denominator vanished

; Exploded/vanished = exploded (N-1)

```

[125] mov esi, 1

```

```

[126] jmp main_path

.check_numerator_vanished:
[127] mov eax, r8d
[128] shr eax, 29 ; Get top 3 bits
[129] cmp eax, 0x01 ; Check for  $\square\square\square$  pattern (vanished positive)
[130] je .check_vanished
[131] cmp eax, 0x06 ; Check for  $\square\square\square$  pattern (vanished negative)
[132] je .check_vanished
[133] jmp main_path

.check_vanished:
[134] mov eax, r10d
[135] shr eax, 29 ; Get top 3 bits
[136] cmp eax, 0x01 ; Check for  $\square\square\square$  pattern (vanished positive)
[137] je .vanished_divide_vanished
[138] cmp eax, 0x06 ; Check for  $\square\square\square$  pattern (vanished negative)
[139] je .vanished_divide_vanished
[140] mov esi, 2
[141] jmp main_path

.denominator_exploded:
; Normal divided by exploded = vanished
[142] mov esi, 2 ; Set flag for vanished (2)
[143] jmp main_path

.denominator_vanished:
; Normal divided by vanished = exploded
[144] mov esi, 1 ; Set flag for exploded (1)
[145] jmp main_path

.exploded_divide_exploded:
; Return exploded divide exploded undefined value
[146] mov eax, 0x00010110 ; EXPLODED_DIVIDE_EXPLODED prefix
[147] mov dword ptr [rdi], eax ; Store real fraction with prefix
[148] mov dword ptr [rdi+4], eax ; Store imaginary fraction with prefix
[149] mov word ptr [rdi+8], 0x8000 ; Set exponent to AMBIGUOUS_EXPONENT
[150] ret

.vanished_divide_vanished:

```

```
; Return vanished divide vanished undefined value
[151] mov eax, 0xf1090000      ; VANISHED_DIVIDE_VANISHED prefix (sign-extended)
[152] mov dword ptr [rdi], eax ; Store real fraction with prefix
[153] mov dword ptr [rdi+4], eax ; Store imaginary fraction with prefix
[154] mov word ptr [rdi+8], 0x8000 ; Set exponent to AMBIGUOUS_EXPONENT
[155] ret

.return_numerator:
; Just copy the numerator as is
[156] mov r8d, dword ptr [rsi] ; Load a (numerator real)
[157] mov r9d, dword ptr [rsi+4] ; Load b (numerator imaginary)
[158] mov di, word ptr [rsi+8] ; Load numerator exponent
[159] mov dword ptr [rdi], r8d ; Store real fraction
[160] mov dword ptr [rdi+4], r9d ; Store imaginary fraction
[161] mov word ptr [rdi+8], di ; Store exponent
[162] ret

.return_denominator:
; Just copy the denominator as is
[163] mov r8d, dword ptr [rdx] ; Load c (denominator real)
[164] mov r9d, dword ptr [rdx+4] ; Load d (denominator imaginary)
[165] mov di, word ptr [rdx+8] ; Load denominator exponent
[166] mov dword ptr [rdi], r8d ; Store real fraction
[167] mov dword ptr [rdi+4], r9d ; Store imaginary fraction
[168] mov word ptr [rdi+8], di ; Store exponent
[169] ret
```

Figure 19: Scalar Bitwise AND Example in Rust

```

let a = Scalar::<i32, i8>::from(42.75); // 101010.11 in binary
let b = Scalar::<i32, i8>::from(27.5);  // 11011.1 in binary

// Bitwise AND (a & b) in the invention:
// Values are aligned based on their exponents:
//   101010.11
//   11011.1
//   -----
//   001010.10
let result = a & b; // Equals 10.5 in decimal, 1010.1 in binary

```

Figure 20: Scalar Bitwise Operations with Signs in Rust

```

// Bitwise operations respect two's complement properties
let pos = Scalar::<i32, i8>::from(42.75); // 101010 in binary
let neg = Scalar::<i32, i8>::from(-27.5); // ~1 00100.1 in binary
// where ~ denotes infinite leading ones

// In two's complement, negative numbers have high bits set to 1
let and_result = pos & neg; // Will retain pos where neg has 1 bits (+32.5)
let or_result = pos | neg;  // Will be negative due to neg's high bits (-17.25)
let xor_result = pos ^ neg; // Flips the bits of pos where neg has 1's (-49.75)
let not_result = !pos;      // Flips all bits (a smidge less than -42.75)

```

Figure 21: Scalar Bit Shift Operations in Rust

```

let value = Scalar::<i32, i8>::from(42);

// Left shift (multiplication by power of 2)
let shifted_left = value << 3; // Equals 336 (42 * 2^3)

// Right shift (division by power of 2)
let shifted_right = value >> 2; // Equals 10.5 (42 / 2^2)

```